

Chapter 3

Solving problems by searching

Objectives

- ❖ Identify the type of agent that solve problem by searching
- ❖ Problem formulation and goal formulation
- ❖ Types of problem based on environment type
- ❖ Discuss various techniques of search strategies
- ❖ Discuss about game playing theory

- **Type of agent that solve problem by searching**
 - Such agent is not reflex or model based reflex agent because this agent needs to achieve some target (goal)
 - It can be goal based or utility based or learning agent
 - Intelligent agent knows that to achieve certain goal, the state of the environment will change sequentially and the change should be towards the goal

•Steps to undertake during searching

- **Problem formulation:**

- Involves:

- Abstracting the real environment configuration into state information using preferred data structure
- Describe the initial state according to the data structure
- deciding the set of all possible action
- The set of action possible on a given state at specific point in the process.
- The cost of the action at each state

- For vacuum world problem, the problem formulation involve:

- State is described as list of 3 elements where the first element describe information about block A, the second element describe information about block B and the last element describe the location of the Agent
- [dirty, dirty, A]
- Suck, moveRight, moveLeft
- Determine which of the above action are valid for a give action
- Cost can be determined in many ways

•Steps to undertake during searching

- **Goal formulation:** refers to the understanding of the objective of the agent based on the state description of the final environment
- For example, for the vacuum world problem, the goal can be formulated as
[clean, Clean, agent at any block]

- **Solution** is a **sequence of world state** in which the final state satisfy the goal or **solution** is **action sequence** in which the last action will result the goal state.
- Each action change one state to the next state of the world
- A search algorithm take a problem as input and returns a solution in the form of action or state sequence.
- To achieve the goal, the action sequences must be executed accordingly
- The general “formulate-search-execute” algorithm of search is given bellow:

Problem-solving agents

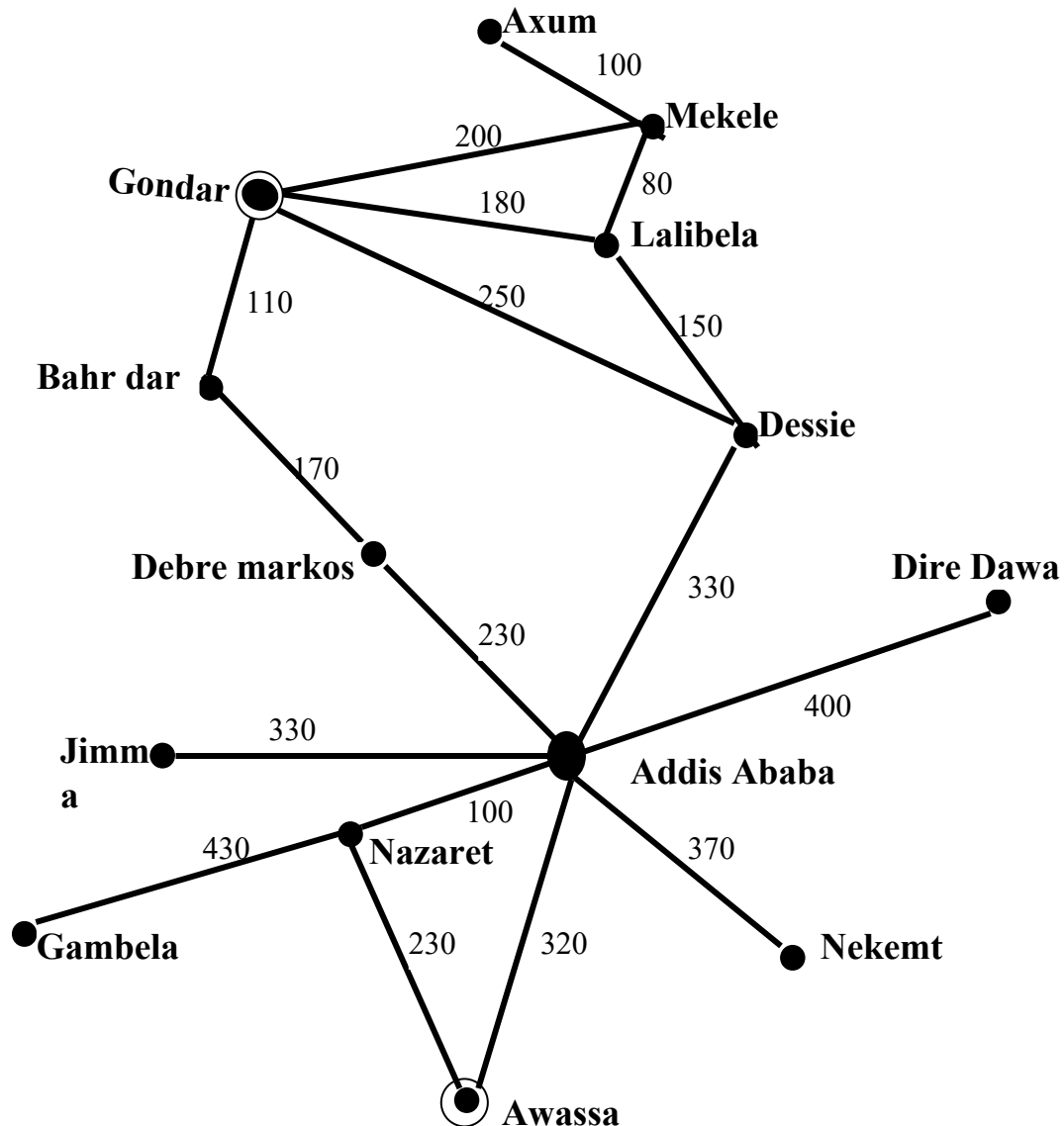
```
function SIMPLE-PROBLEM-SOLVING-AGENT(percept) returns an action
  static: seq, an action sequence, initially empty
          state, some description of the current world state
          goal, a goal, initially null
          problem, a problem formulation

  state ← UPDATE-STATE(state, percept)
  if seq is empty then do
    goal ← FORMULATE-GOAL(state)
    problem ← FORMULATE-PROBLEM(state, goal)
    Seq ← SEARCH(problem, goal)
  action ← FIRST(seq)
  seq ← REST(seq)
  return action
```

Agent Program

```
function SimpleAgent( problem, environment, currentState, goalState)  
    percept ← GETPERCEPT(sensor, environment)  
    while currentState != goalState  
        action ← SIMPLE_PROBLEM_SOLVING_AGENT(percept)  
        CurrentState ← UPDATE_STATE(currentState, action)
```

Example: Road map of Ethiopia



Example: Road map of Ethiopia

- Current position of the agent: Awassa.
- Needs to arrive to: Gondar
- Formulate goal:
 - be in Gondar
- Formulate problem:
 - **states**: various cities
 - **actions**: drive between cities
- Find solution:
 - sequence of cities, e.g., Awassa, Nazarez, Addis Ababa, Dessie, Gondar

Types of Problems

- Four types of problems exist in the real situations:
 - 1. single-state problem**
 - The environment is **Deterministic** and **fully observable**
 - Out of the possible state space, agent knows exactly which state it will be in; solution is a sequence
 - 2. sensor less problem (conformant problem)**
 - The environment is **non-observable**
 - It is also called **multi-state problem**
 - Agent may have no idea where it is; solution is a sequence

Types of Problems

3. contingency problem

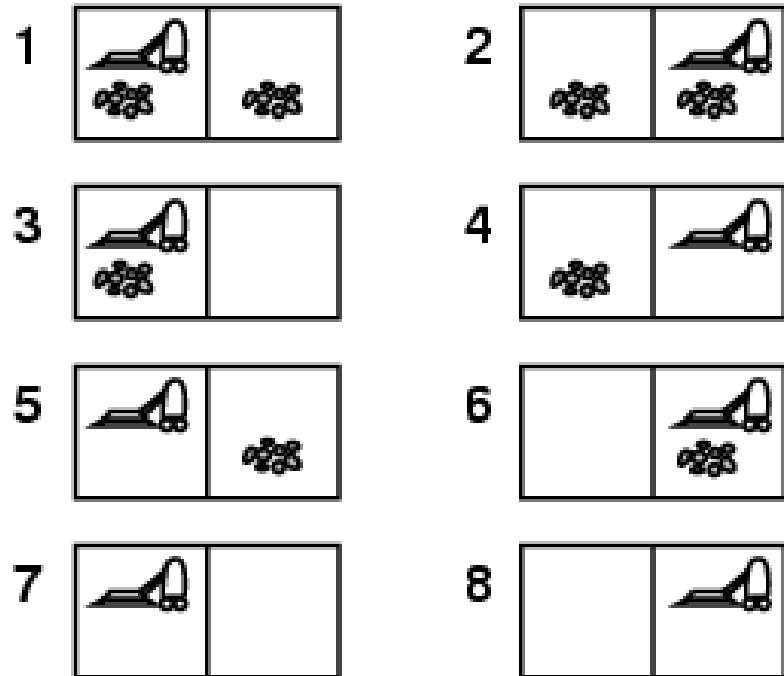
- The environment is **nondeterministic** and/or **partially observable**
- It is not possible to know the effect of the agent action
- percepts provide **new** information about current state

4. exploration problem

- The environment is partially observable
- It is also called **unknown state space**

Example: vacuum world

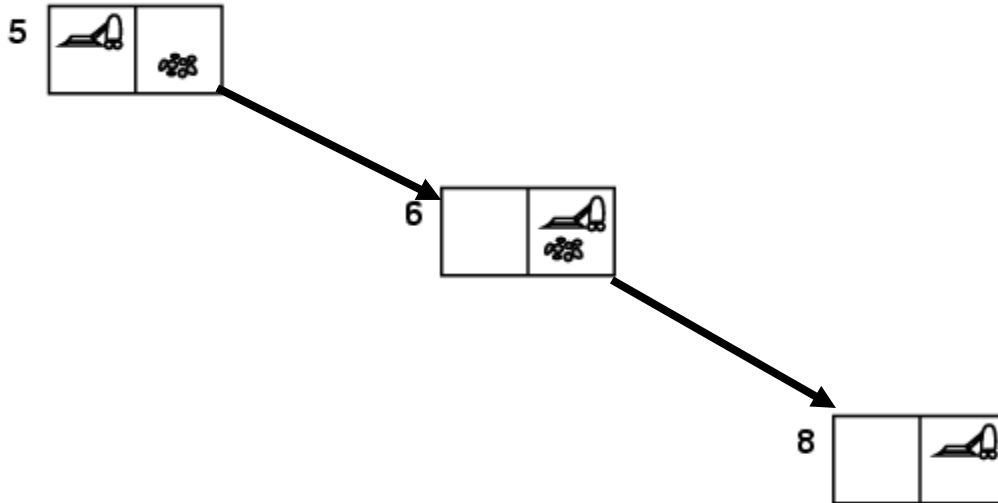
- Single-state
 - Starting state us known say in #5.
 - What is the Solution?



Example: vacuum world

- Single-state, start in #5.

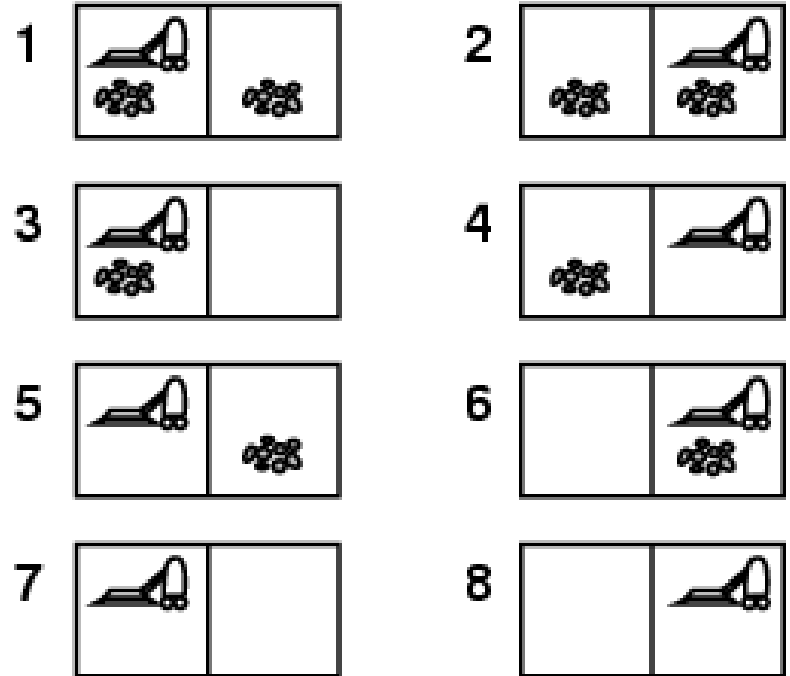
Solution? [*Right, Suck*]



Example: vacuum world

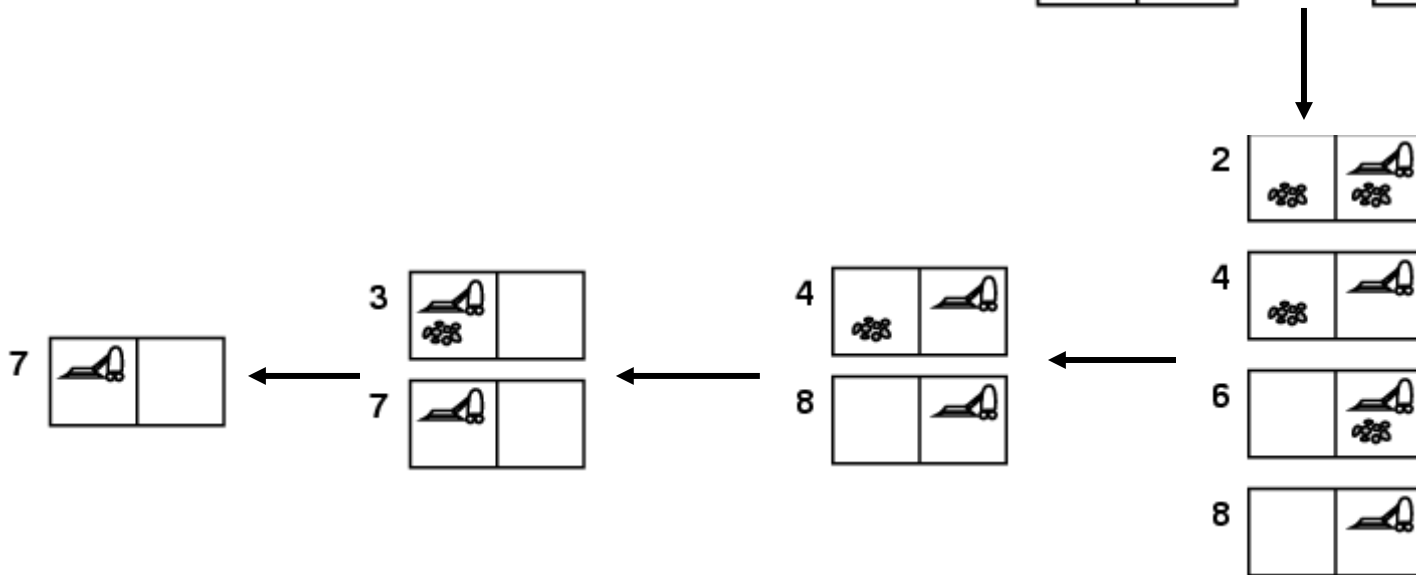
- Sensorless,

- It doesn't know what the current state is
- So the current start is either of the following: $\{1, 2, 3, 4, 5, 6, 7, 8\}$
- *What is the Solution?*



Example: vacuum world

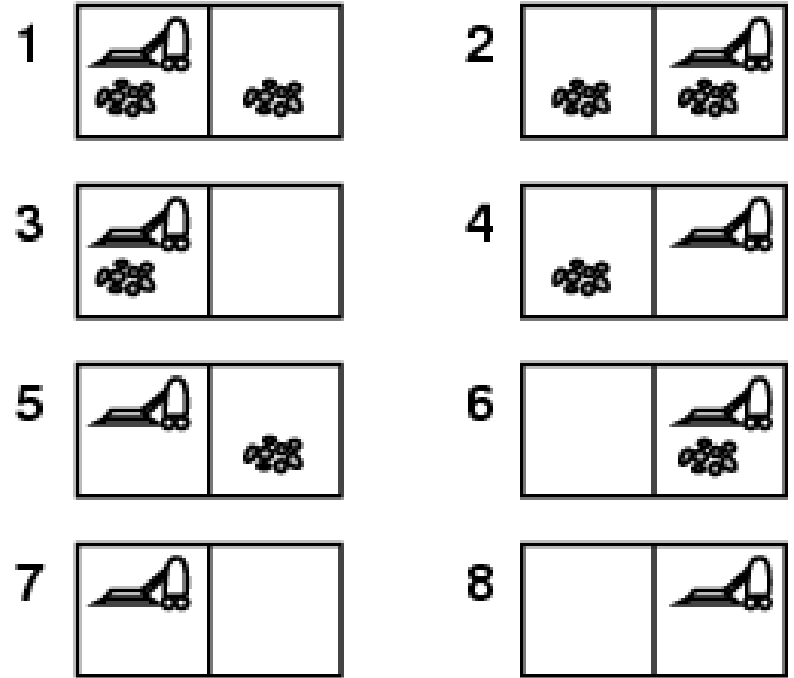
- Sensorless Solution
- *Right* goes to $\{2, 4, 6, 8\}$
- Solution?
- $[Right, Suck, Left, Suck]$



Example: vacuum world

- Contingency

- Nondeterministic: *Suck* may dirty a clean carpet
- Partially observable:
- Hence we have partial information
- Let's assume the current percept is: $[L, \text{Clean}]$
- i.e. start in #5 or #7

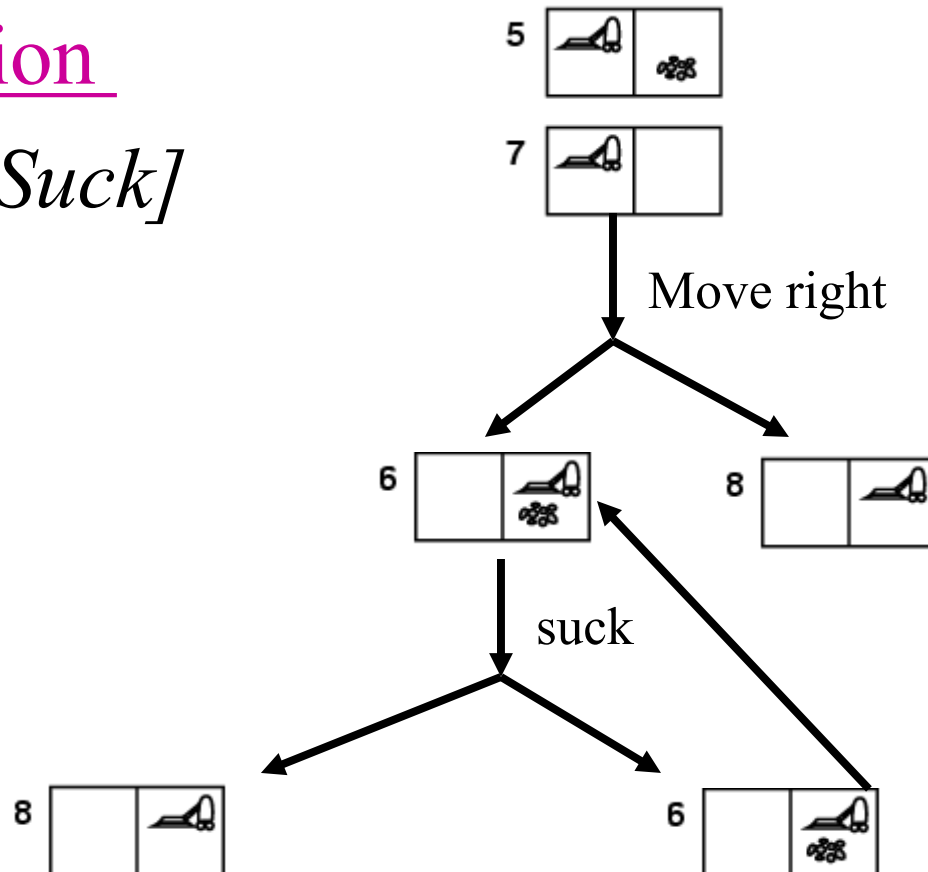


- What is the Solution?

Example: vacuum world

- Contingency Solution

[Right, if dirt then Suck]



Exploration

Example 1:

- Assume the agent is some where outside the blocks and wants to clean the block. So how to get into the blocks? No clear information about their location
- What will be the solution?
- Solution is exploration

Example 2:

- The agent is at some point in the world and want to reach a city called CITY which is unknown to the agent.
- The agent doesn't have any map
- What will be the solution?
 - Solution is exploration

Some more problems that can be solved by searching

- We have seen two such problems: The road map problem and the vacuum cleaner world problem
- The following are some more problems
 - The three mice and the three cats problem
 - The three cannibal and the three missionaries problem
 - The water jug problem

3 cat and 3 mice puzzle

- Three cat and three mice come to a crocodile infested river. There is a boat on their sides that can be used by one or two “persons”. If cats outnumber the mice at any time, the cats eat the mice. How can they use the boat to cross the river so that all mice survive.
- State description
 - [#of cats to the left side,
#of mice to the left side,
boat location,
#of cats to the right side,
#of mice to the right side]
- Initial state
 - [3,3,Left,0,0]
- Goal
 - [0,0,Right,3,3]

3 cat and 3 mice puzzle

- Action
 - A legal action is a move which moves upto two person at a time using the boat from the boat location to the other side provided that action doesn't contradict the constraint ($\#cats \leq \#mice$)
 - We can represent the action as
 - Move_Ncats_M_mice_lr if boat is at the left side or
 - Move_Ncats_M_mice_rl if boat is at the right side.
 - All the set of possible action except the constraints are:

Move-1m1c-lr

Move-2m-lr

Move-1m1c-rl

Move-1c-rl

Move-2c-rl

Move-1c-lr

Move-2c-lr

Move-1m-rl

Move-2m-rl

Move-1m-lr

3 cat and 3 mice puzzle

- Question
 - Draw the state space of the problem
 - Provide one possible solution

Move-2c-lr

Move-1c-rl

Move-2c-lr

Move-1c-rl

Move-2m-lr

Move-1m1c-rl

Move-2m-lr

Move-1c-rl

Move-2c-lr

Move-1c-rl

Move-2c-lr

3 cannibal and 3 missionaries problem

- Three missionaries and three cannibals come to the bank of a river they wish to cross.
- There is a boat that will hold only two and any of the group is able to row.
- If there are ever more missionaries than cannibals on any side of the river the cannibals will get converted.
- How can they use the boat to cross the river without conversion.
- State description
 - [#of cannibals to the left side,
#of missionaries to the left side,
boat location,
#of cannibals to the right side,
#of missionaries to the right side]
- Initial state
 - [3,3,Left,0,0]
- Goal
 - [0,0,Right,3,3]

3 cannibal and 3 missionaries problem

- Action
 - A legal action is a move which moves up to two person at a time using the boat from the boat location to the other side provided that action doesn't contradict the constraint ($\#cannibal < \#missionaries$)
 - We can represent the action as
 - Move_Ncannibal_M_missionaries_lr if boat is at the left side or
 - Move_Ncannibal_M_missionaries_rl if boat is at the right side.
 - All the set of possible action except the constraints are:

Move-1m1c-lr

Move-1m1c-rl

Move-2c-rl

Move-2c-lr

Move-2m-rl

Move-2m-lr

Move-1c-rl

Move-1c-lr

Move-1m-rl

Move-1m-lr

3 cannibal and 3 missionaries problem

- Question
 - Draw the state space of the problem
 - Provide one possible solution

Move-2c-lr

Move-1c-rl

Move-2c-lr

Move-1c-rl

Move-2m-lr

Move-1m1c-rl

Move-2m-lr

Move-1c-rl

Move-2c-lr

Move-1c-rl

Move-2c-lr

Water Jug problem

- We have one 3 liter jug, one 5 liter jug and unlimited supply of water. The goal is to get exactly one liter of water in either of the jug. Either jug can be emptied, filled or poured into the other.
- State description
 - [Amount of water in 5 liter jug,
Amount of water in 3 liter jug]
- Initial state
 - [0,0]
- Goal
 - [1,ANY] or [ANY, 1]

Water Jug problem

- Action
 - Fill the 3 litter jug with water (F_3)
 - Fill the 5 litter jug with water(F_5)
 - Empty the 5 litter jug (E_5)
 - Empty the 3 litter jug (E_3)
 - Pour the all 3 litter jug water onto the 5 litter jug ($P_{3 \rightarrow 5}$)
 - Pour the all 5 litter jug water onto the 3 litter jug ($P_{5 \rightarrow 3}$)
 - Pour the 3 litter jug water onto the 5 litter jug until the 5 litter jug filled completely. ($P_part_{3 \rightarrow 5}$)
 - Pour the 5 litter jug water onto the 3 litter jug until the 3 litter jug filled completely. ($P_part_{5 \rightarrow 3}$)

Water Jug problem

- Question
 - Draw the complete state space diagram
 - Find one possible solution as action and state sequence

<i>Initial state</i>	<i>[0,0]</i>
<u>Action</u>	<u>State</u>
<i>F_3</i>	<i>[3,0]</i>
<i>$P_{3 \rightarrow 5}$</i>	<i>[0,3]</i>
<i>F_3</i>	<i>[3,3]</i>
<i>$P_{part_{3 \rightarrow 5}}$</i>	<i>[1,5]</i>

Well-defined problems and solutions (single state)

- A well defined problem is a problem in which
 - The start state of the problem
 - Its goal state
 - The possible actions (operators that can be applied to make move from state to state)
 - The constraints upon the possible action to avoid invalid moves (this defines legal and illegal moves)are known in advance

Well-defined problems and solutions (single state)

- A problem which is not well defined is called ill-defined
 - Ill-defined problem presents a dilemma in planning to get the goal
 - The goal may not be precisely formulated
 - Examples
 - Cooking dinner
 - Writing term paper
- All the problem we need to consider in this course are well defined

Well-defined problems and solutions

A **problem** is defined by four items:

1. **initial state** e.g., "at Awassa"
2. **actions** or **successor function** $S(x)$ = set of **action–state** pairs
 - e.g., $S(Awassa) = \{ \langle Awassa \rightarrow Addis\ Ababa, Addis\ Ababa \rangle, \langle Awassa \rightarrow Nazareth, Nazareth \rangle, \dots \}$
 - Note: $\langle A \rightarrow B, B \rangle$ indicates action is $A \rightarrow B$ and next state is B
3. **goal test**, can be
 - **explicit**, e.g., $x = \text{"at Gonder"}$
 - **implicit**, e.g., $CheckGoal(x)$

Well-defined problems and solutions (single state)

4. The constraints

path cost (additive)

- e.g., sum of distances, number of actions executed, etc.
- $c(x, a, y)$ is the step cost, assumed to be ≥ 0 (the cost of applying action a being at initial state x which takes into next state y)

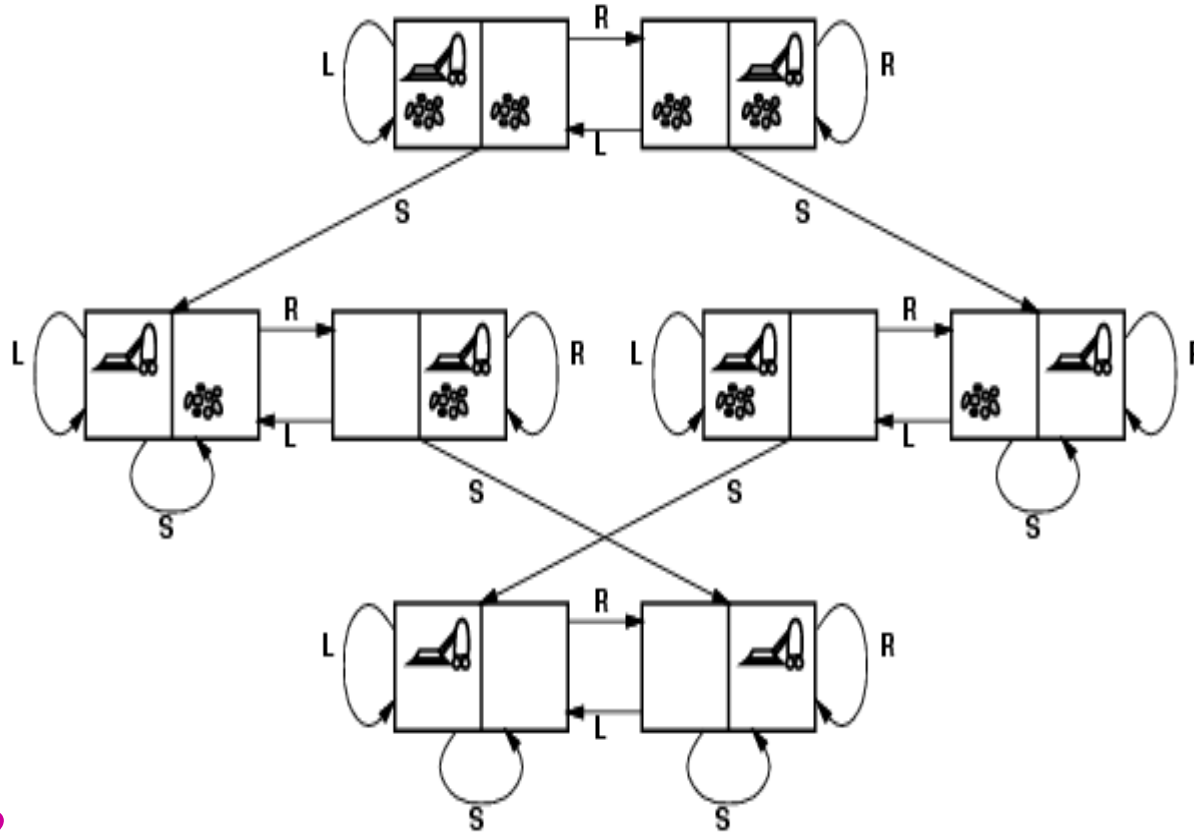
invalid (actions that doesn't change states)

solution is a sequence of actions leading from the initial state to a goal state or sequence of states in which the last state is the goal state

Selecting a state space

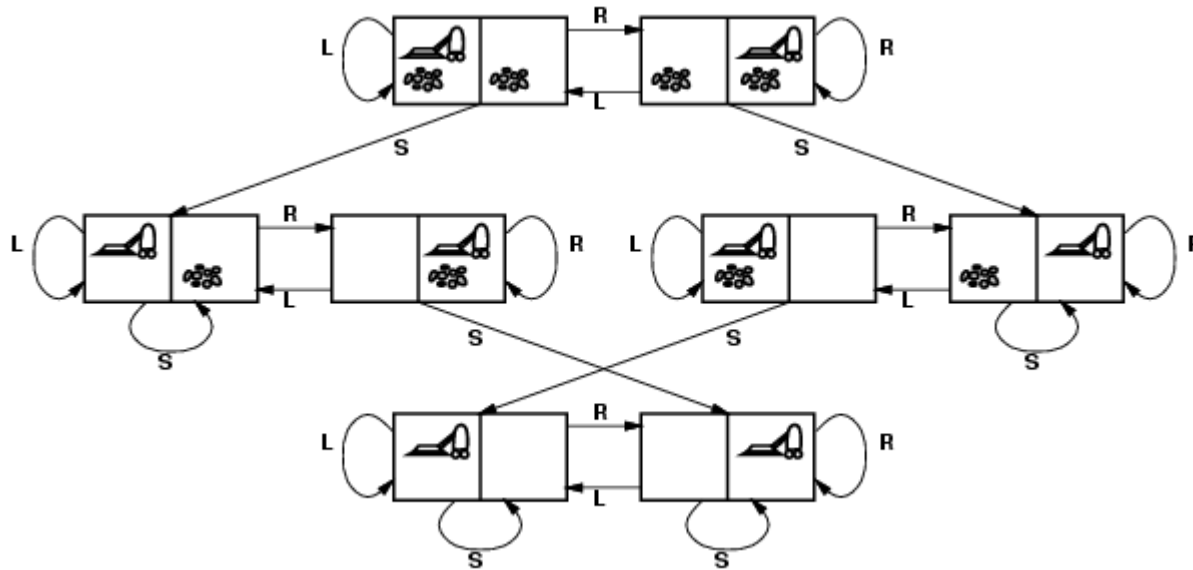
- Real world problem can not be directly represented in the agent architecture since it is absurdly complex
 - state space must be **abstracted** for problem solving
- (Abstract) state = set of real states
- (Abstract) action = complex combination of real actions
 - e.g., “Awasa → Addis Ababa” represents a complex set of possible routes, detours, rest stops, etc.
- (Abstract) solution = set of real paths that are solutions in the real world
- Each abstract action should be "easier" than the original problem

Vacuum world state space graph



- states?
- actions?
- goal test?
- path cost?

Vacuum world state space graph



- states? Information on dirt and robot location (one of the 8 states)
- actions? *Left, Right, Suck*
- goal test? no dirt at all locations
- path cost? 1 per action

Example: The 8-puzzle

7	2	4
5		6
8	3	1

Start State

	1	2
3	4	5
6	7	8

Goal State

- states?
- actions?
- goal test?
- path cost?

Example: The 8-puzzle

7	2	4
5		6
8	3	1

Start State

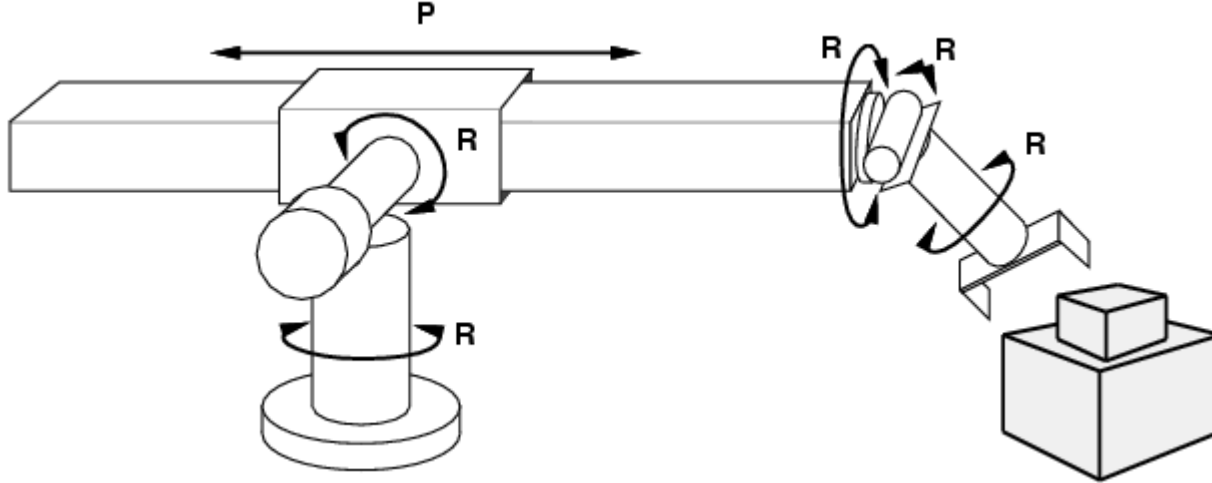
	1	2
3	4	5
6	7	8

Goal State

- states? locations of tiles
- actions? move blank left, right, up, down
- goal test? = goal state (given)
- path cost? 1 per move

[Note: optimal solution of n -Puzzle family is NP-hard]

Example: robotic assembly



- states?: real-valued coordinates of robot joint angles and parts of the object to be assembled
- actions?: continuous motions of robot joints
- goal test?: complete assembly
- path cost?: time to execute

Searching For Solution (Tree search algorithms)

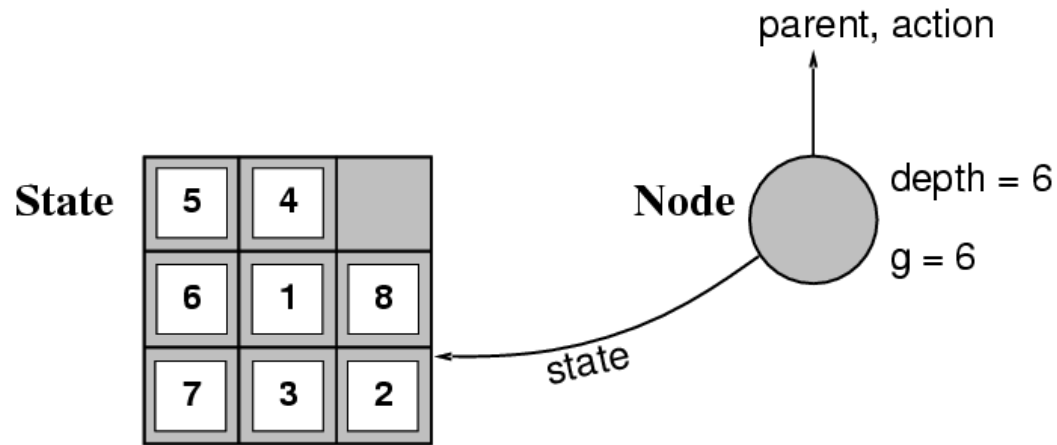
- Given state space, and network of states via actions.
- The network structure is usually a graph
- Tree is a network in which there is exactly one path defined from the root to any node
- Given state S and valid actions being at S
 - the set of next state generated by executing each action is called successor of S
- Searching for solution is a simulated exploration of state space by generating successors of already-explored states

Implementation issue: states vs. nodes

- A **state** is a (representation of) a physical configuration
- A **node** is a data structure constituting part of a search tree
 - It includes:
 - **state**,
 - **parent node**,
 - **action**,
 - **depth and**
 - **one or more costs** [like path cost $g(x)$, *heuristic cost* $h(x)$, *evaluation function cost* $f(x)$]

Implementation issue: states vs. nodes

- Example

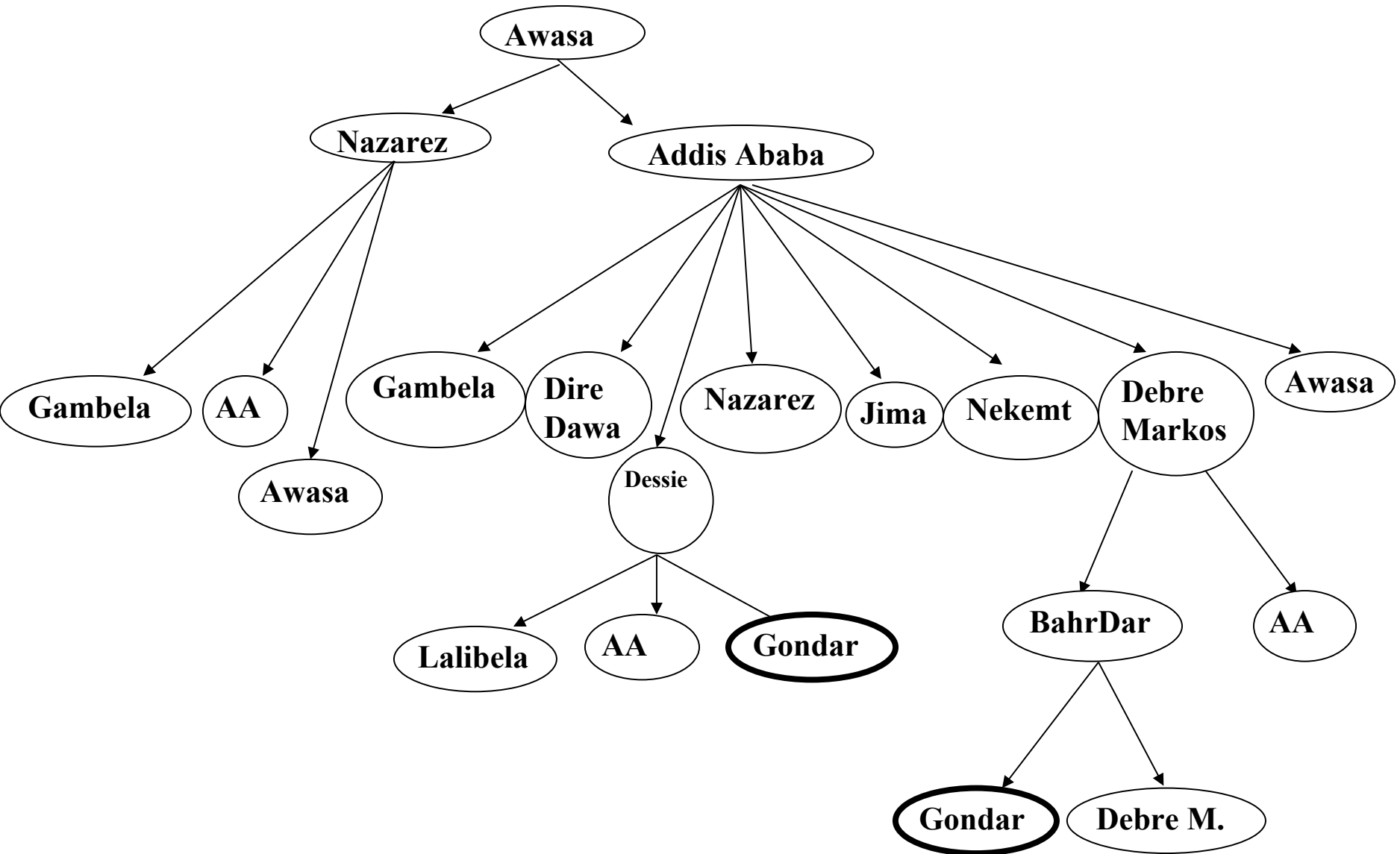


Searching For Solution (Tree search algorithms)

```
function TREE-SEARCH(problem, strategy) returns a solution, or failure
  initialize the search tree using the initial state of problem
  loop do
    if there are no candidates for expansion then return failure
    choose a leaf node for expansion according to strategy
    if the node contains a goal state then return the corresponding solution
    else expand the node and add the resulting nodes to the search tree
```

- The Successor-Fn generate all the successors state and the action that leads moves the current state into the successor state
- The Expand function creates new nodes, filling in the various fields of the node using the information given by the Successor-Fn and the input parameters

Tree search example



Implementation: general tree search

```
function TREE-SEARCH(problem, fringe) returns a solution, or failure  
  fringe  $\leftarrow$  INSERT(MAKE-NODE(INITIAL-STATE[problem]), fringe)  
  loop do  
    if fringe is empty then return failure  
    node  $\leftarrow$  REMOVE-FRONT(fringe)  
    if GOAL-TEST[problem](STATE[node]) then return SOLUTION(node)  
    fringe  $\leftarrow$  INSERTALL(EXPAND(node, problem), fringe)
```

```
function EXPAND(node, problem) returns a set of nodes  
  successors  $\leftarrow$  the empty set  
  for each action, result in SUCCESSOR-FN[problem](STATE[node]) do  
    s  $\leftarrow$  a new NODE  
    PARENT-NODE[s]  $\leftarrow$  node; ACTION[s]  $\leftarrow$  action; STATE[s]  $\leftarrow$  result  
    PATH-COST[s]  $\leftarrow$  PATH-COST[node] + STEP-COST(node, action, s)  
    DEPTH[s]  $\leftarrow$  DEPTH[node] + 1  
    add s to successors  
  return successors
```

Search strategies

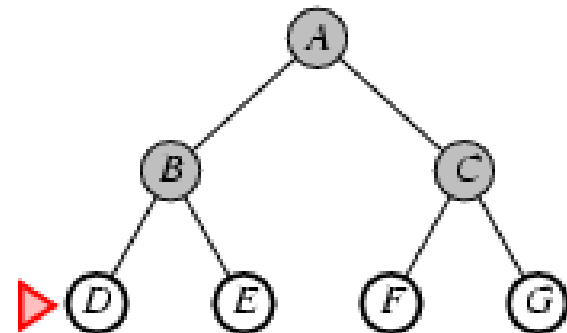
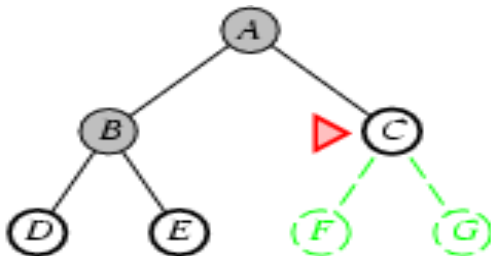
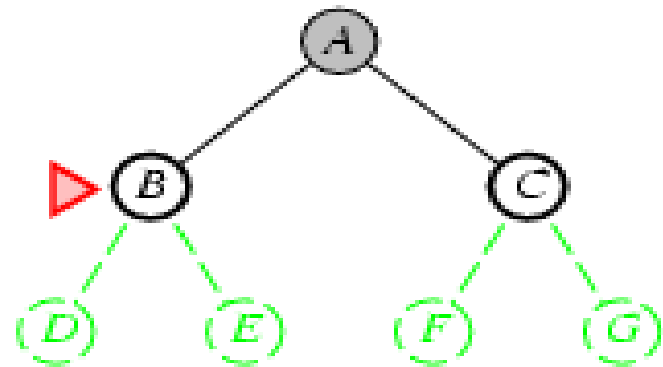
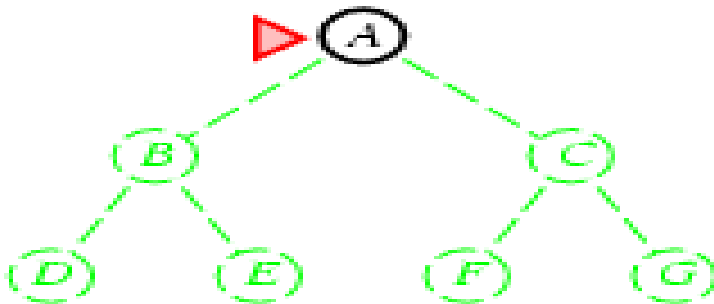
- A search strategy is defined by picking the **order of node expansion**
- Strategies are evaluated along the following dimensions:
 - **completeness**: does it always find a solution if one exists?
 - **time complexity**: number of nodes generated
 - **space complexity**: maximum number of nodes in memory
 - **optimality**: does it always find a least-cost solution?
- Time and space complexity are measured in terms of
 - b : maximum branching factor of the search tree
 - d : depth of the least-cost solution
 - m : maximum depth of the state space (may be ∞)
- Generally, searching strategies can be classified in to two as **uninformed** and **informed** search strategies

Uninformed search (blind search) strategies

- **Uninformed** search strategies use only the information available in the problem definition
- They have no information about the number of steps or the path cost from the current state to the goal
- They can distinguish the goal state from other states
- They are still important because there are problems with no additional information.
- Six kinds of such search strategies will be discussed and each depends on the order of expansion of successor nodes.
 1. Breadth-first search
 2. Uniform-cost search
 3. Depth-first search
 4. Depth-limited search
 5. Iterative deepening search
 6. Bidirectional search

Breadth-first search

- Expand shallowest unexpanded node
- Finds the shallowest goal state
- **Implementation:**
 - *Fringe (open list)* is a FIFO queue, i.e., new successors go at the end

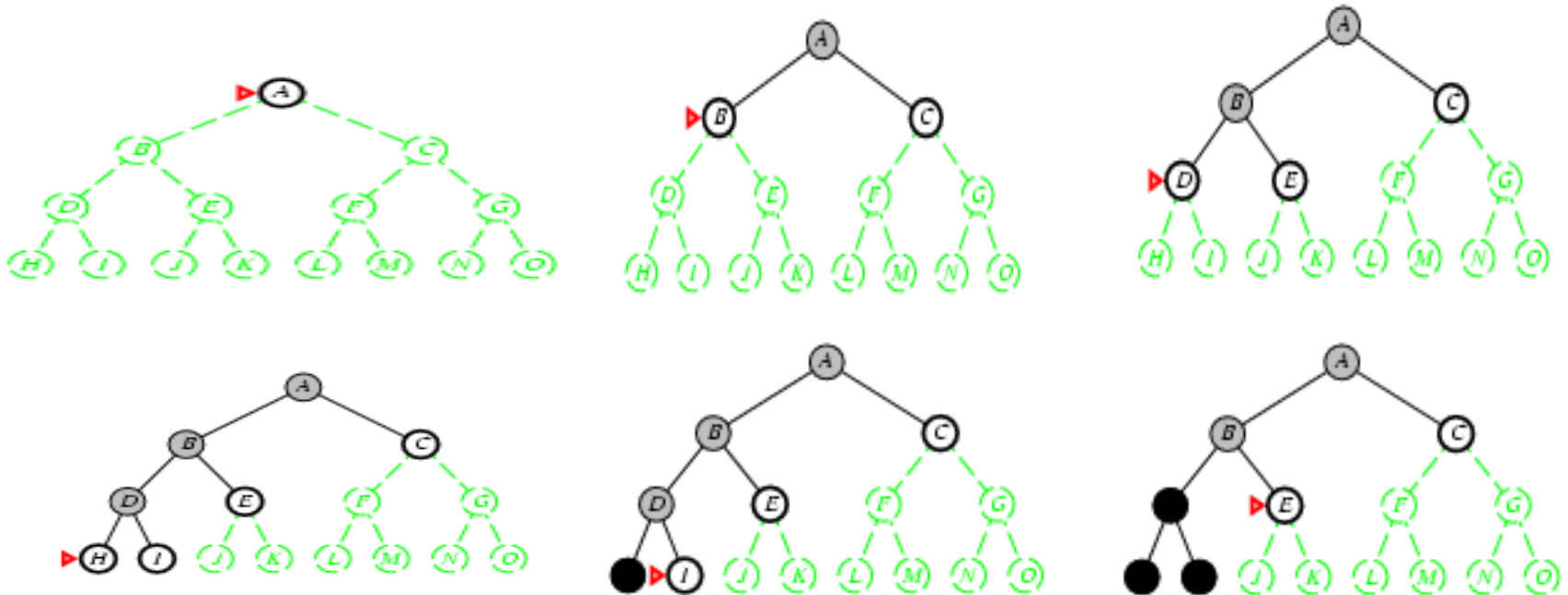


Properties of breadth-first search

- Complete? Yes (if b is finite, which is true in most cases)
- Time? $1 + b + b^2 + b^3 + \dots + b^d = O(b^{d+1})$
 - at depth value = i , there are b^i nodes expanded for $i \leq d$
- Space? $O(b^d)$ (keeps every node in memory)
 - a maximum of this match node will be while reaching to the goal node
 - This is a major problem for real problem
- Optimal? Yes (if cost = constant (k) per step)
- **Space** is the bigger problem (more than time)

Depth-first search

- Expand deepest unexpanded node
- Implementation:
 - *fringe* = LIFO queue, i.e., put successors at front



Properties of depth-first search

- Complete? No: fails in infinite-depth spaces, spaces with loops
 - Modify to avoid repeated states along path (see graph search)
 - complete in finite spaces
- Time? $O(b^m)$: terrible if m is much larger than d
 - but if solutions are dense, may be much faster than breadth-first
- Space? $O(bm)$, i.e., linear space!
 - When the strategy move one step down the tree, it will add m nodes into the fringe and will have bm nodes at the worst case.
- Optimal? No

Depth-limited search

- We can see that the breadth search is complete which can be taken as its advantage though its space complexity is the worst
- Similarly the depth first search strategy is best in terms of space complexity even if it is the worst in terms of its completeness and time complexity compared to breadth first search
- Hence, we can find an algorithm that incorporate both benefits and avoid the limitation
- Such algorithm is called depth limited search and its improved version is called iterative deepening search strategy.
- These two strategies will be explored in the following sections

Depth-limited search

- Depth-first search with depth limit l , will truncate all nodes having depth value greater than l from the search space and apply depth first search on the rest of the structure
- It return **solution** if solution exist, if there is no solution
- it return **cutoff** if $l < m$, **failure** otherwise

Recursive implementation:

```
function DEPTH-LIMITED-SEARCH(problem, limit) returns soln/fail/cutoff
    RECURSIVE-DLS(MAKE-NODE(INITIAL-STATE[problem]), problem, limit)

function RECURSIVE-DLS(node, problem, limit) returns soln/fail/cutoff
    cutoff-occurred?  $\leftarrow$  false
    if GOAL-TEST[problem](STATE[node]) then return SOLUTION(node)
    else if DEPTH[node] = limit then return cutoff
    else for each successor in EXPAND(node, problem) do
        result  $\leftarrow$  RECURSIVE-DLS(successor, problem, limit)
        if result = cutoff then cutoff-occurred?  $\leftarrow$  true
        else if result  $\neq$  failure then return result
    if cutoff-occurred? then return cutoff else return failure
```

Depth-limited search

- Complete? No (fail if all solution exist at depth $> l$)
- Time? $O(b^l)$
- Space? $O(bl)$
- Optimal? No

Iterative deepening search

- Depth limit search never return a solution due to the limitation of the limit if all solution node exist at $\text{depth} > 1$. This limitation can be avoided by applying iterative deepening search strategy
- **Prototype**

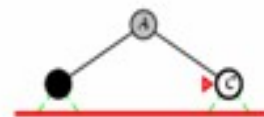
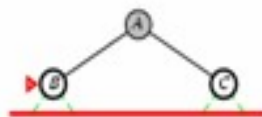
```
function ITERATIVE-DEEPENING-SEARCH(problem) returns a solution, or fail-  
ure  
  inputs: problem, a problem  
  for depth  $\leftarrow 0$  to  $\infty$  do  
    result  $\leftarrow$  DEPTH-LIMITED-SEARCH(problem, depth)  
    if result  $\neq$  cutoff then return result
```

Iterative deepening search / =0

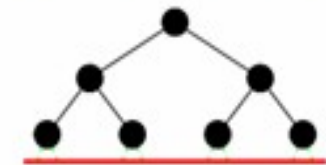
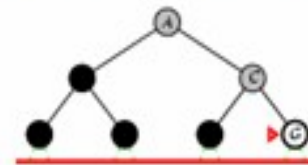
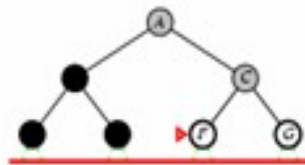
Limit = 0



Limit = 1

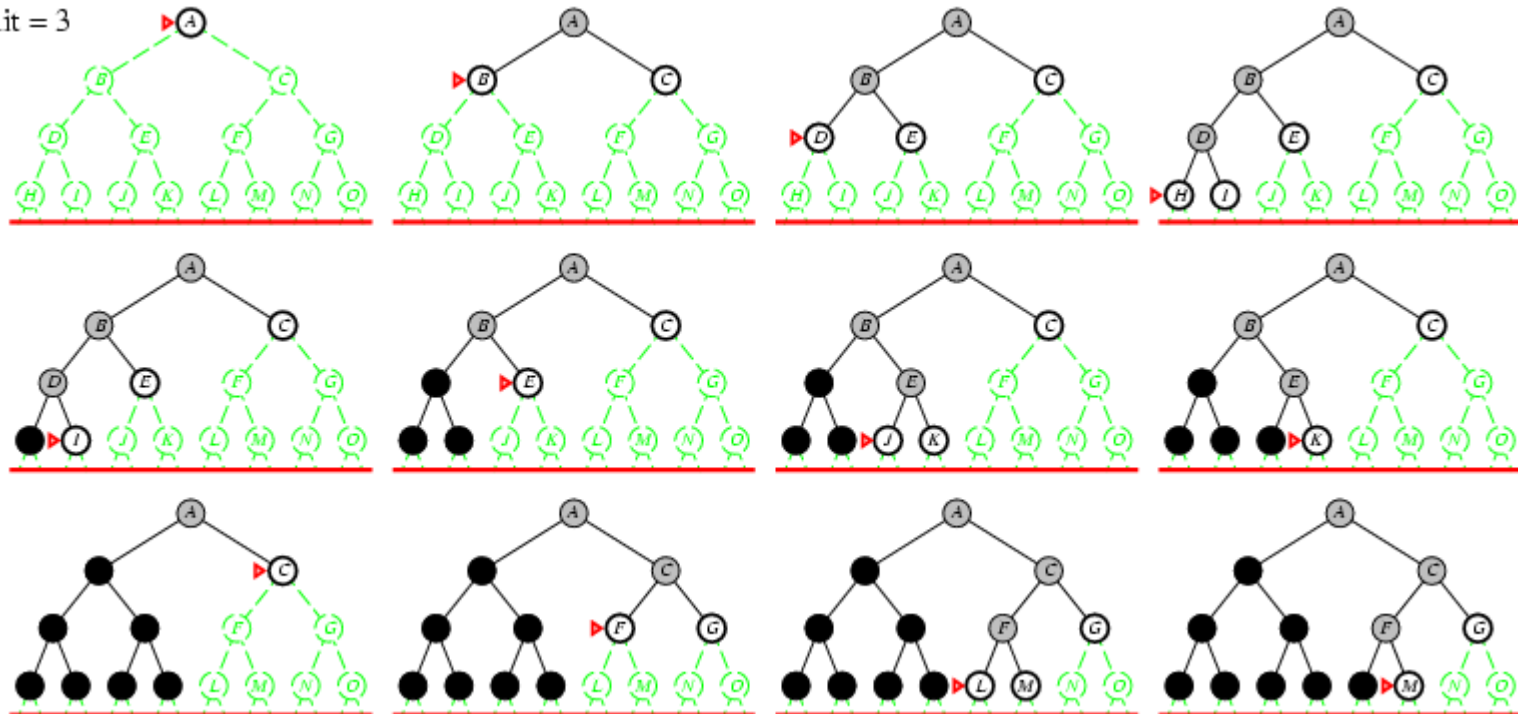


Limit = 2



Iterative deepening search / =3

Limit = 3

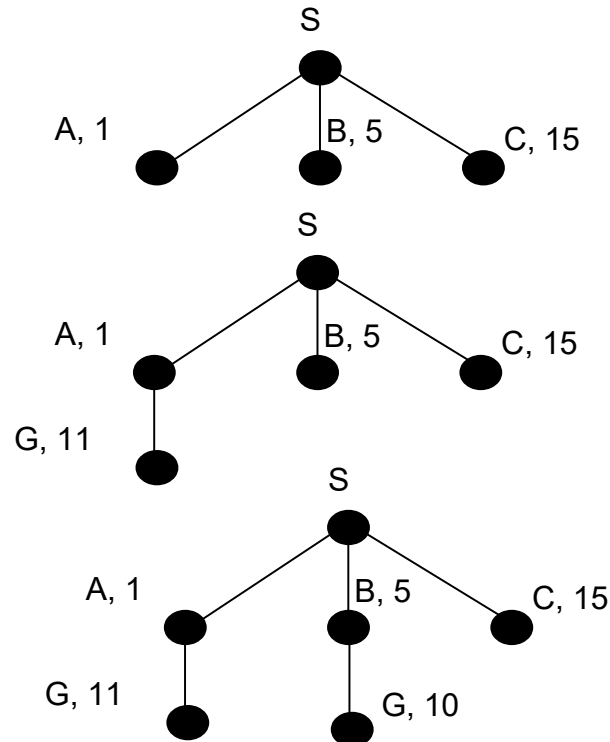
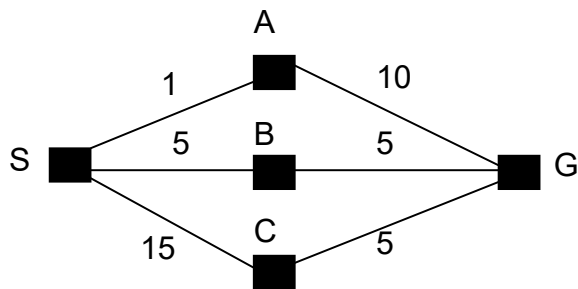


Properties of iterative deepening search

- Complete? Yes
- Time? $(d+1)b^0 + d b^1 + (d-1)b^2 + \dots + b^d = O(b^d)$
- Space? $O(bd)$
- Optimal? Yes, if step cost = 1

Uniform-cost search

- Expand least-cost unexpanded node
- **Implementation:**
 - *fringe* = queue ordered by path cost
- Equivalent to breadth-first if step costs all equal
- Consider the problem that moves from node S to G



Uniform-cost search

- It finds the cheapest solution if the cost of a path never decrease as we go along the path
- i.e. $g(\text{sucessor}(n)) \geq g(n)$ for every node n .
- Complete? Yes
- Time? # of nodes with $g \leq \text{cost of optimal solution}$
 - let ϵ be the minimum step cost in the search tree, C^* is the total cost of the optimal solution and branching factor b .
 - Now you can ask, In the worst case, what will be the number of nodes that exist all of which has step cost = ϵ , branching factor b and path cost $\leq C^*$. The resulting tree will have depth value $\text{floor}(C^*/\epsilon)$
 - Hence the total node will be $b^{\text{ceilling}(C^*/\epsilon)}$. Therefore, time complexity becomes $O(b^{\text{ceiling}(C^*/\epsilon)})$
- Space? # of nodes with $g \leq \text{cost of optimal solution}$, $O(b^{\text{ceiling}(C^*/\epsilon)})$
- Optimal? Yes – nodes expanded in increasing order of $g(n)$

Bidirectional search

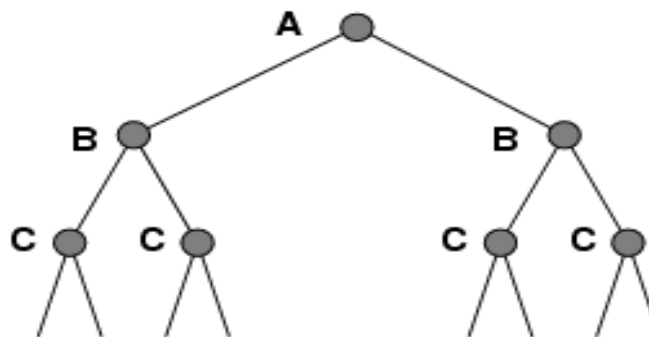
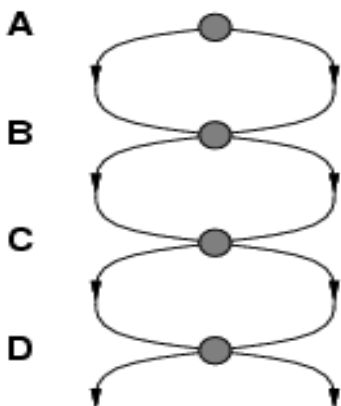
- Simultaneously search forward from the initial state and backward from the goal state and terminate when the two search meet in the middle
- Reconstruct the solution by backward tracking towards the root and forward tracking towards the goal from the point of intersection
- These algorithm is efficient if there any very limited one or two nodes with solution state in the search space

Summary of algorithms

Criterion	Breadth-First	Uniform-Cost	Depth-First	Depth-Limited	Iterative Deepening
Complete?	Yes	Yes	No	No	Yes
Time	$O(b^{d+1})$	$O(b^{\lceil C^*/\epsilon \rceil})$	$O(b^m)$	$O(b^l)$	$O(b^d)$
Space	$O(b^{d+1})$	$O(b^{\lceil C^*/\epsilon \rceil})$	$O(bm)$	$O(bl)$	$O(bd)$
Optimal?	Yes	Yes	No	No	Yes

Repeated states

- Failure to detect repeated states can turn a linear problem into an exponential one!



Graph search

```
function GRAPH-SEARCH(problem, fringe) returns a solution, or failure
  closed ← an empty set
  fringe ← INSERT(MAKE-NODE(INITIAL-STATE[problem]), fringe)
  loop do
    if fringe is empty then return failure
    node ← REMOVE-FRONT(fringe)
    if GOAL-TEST[problem](STATE[node]) then return SOLUTION(node)
    if STATE[node] is not in closed then
      add STATE[node] to closed
      fringe ← INSERTALL(EXPAND(node, problem), fringe)
```

Informed search algorithms

- Informed search is a strategy that uses information about the cost that may incur to achieve the goal state from the current state.
- The information may not be accurate. But it will help the agent to make better decision
- This information is called heuristic information

- There several algorithms that belongs to this group. Some of these are:
 - Best-first search
 1. Greedy best-first search
 2. A* search
 - Memory Bound Best First search
 1. Iterative deepening A* (IDA*) search
 2. Simplified Memory –Bounded A* (SMA*) search
 - Iterative improvement algorithm (Local search algorithms)
 1. Hill-climbing search
 2. Simulated annealing search
 - Genetic algorithms

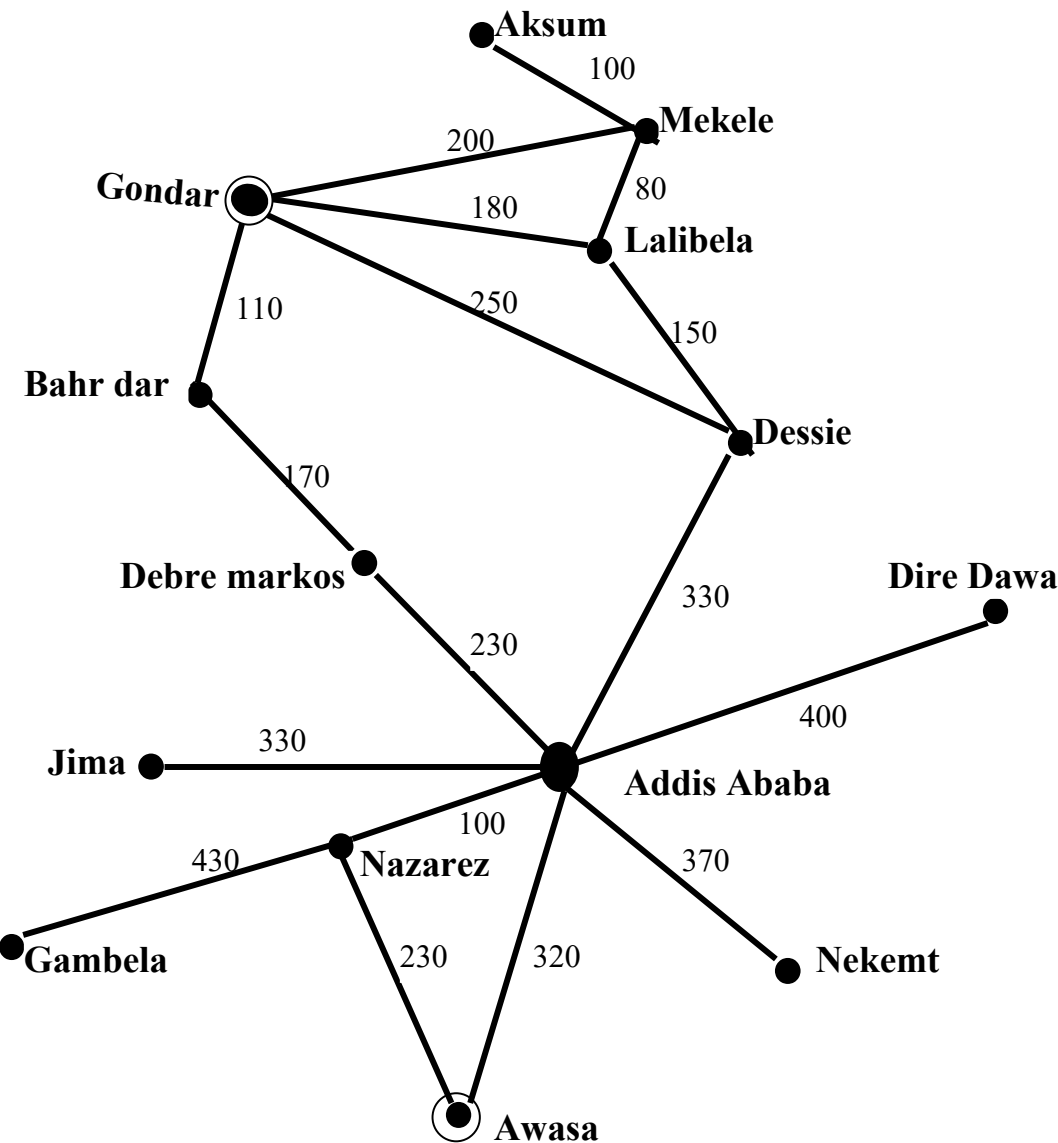
Best-first search

- ❖ Idea: use an **evaluation function** $f(n)$ for each node
 - ❖ Estimate of "desirability" using heuristic and path cost
 - ❖ Expand most desirable unexpanded node
- ❖ The information gives a clue about which node to be expanded first
- ❖ This will be done during queuing
- ❖ The best node according to the evaluation function may not be best

Implementation:

- ❖ Order the nodes in fringe in decreasing order of desirability (increasing order of cost evaluation function)

Ethiopia Map with step costs in km



Straight Line distance to Gondar

Gondar	0
Aksum	100
Mekele	150
Lalibela	110
Desseie	210
Bahr dar	90
Debre Markos	170
Addis Ababa	321
Jima	300
Diredawa	350
Nazarez	340
Gambela	410
Awasa	500
Nekemt	420

Greedy best-first search

- Evaluation function $f(n) = h(n)$ (**h**euristic)
- = estimate of cost from n to *goal*
- That means the agent prefers to choose the action which is assumed to be best after every action
- e.g., $h_{SLD}(n)$ = straight-line distance from n to Gondar
- Greedy best-first search expands the node that **appears** to be closest to goal (It tries to minimize the estimated cost to reach the goal)

Example One

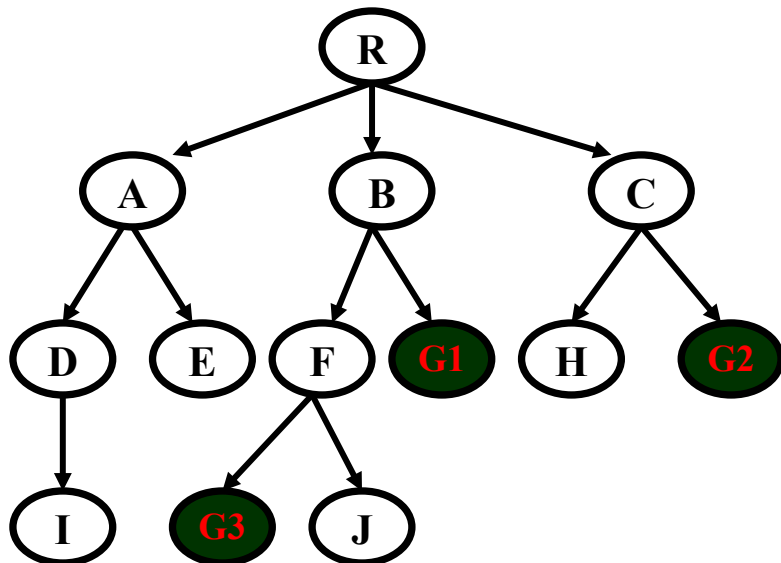
Greedy best-first search example

Show the flow to move from Awasa to Gondar using the given road map graph

Example Two

Greedy best-first search example

- Given the following tree structure, show the content of the open list and closed list generated by Greedy best first search algorithm



Heuristic

R	→	G	-----	100
A	→	G	-----	60
B	→	G	-----	80
C	→	G	-----	70
D	→	G	-----	65
E	→	G	-----	40
F	→	G	-----	45
H	→	G	-----	10
I	→	G	-----	20
J	→	G	-----	8
G ₁ , G ₂ , G ₃ → G				----- 0

Properties of greedy best-first search

- Complete? Yes if repetition is controlled otherwise it can get stuck in loops
- Time? $O(b^m)$, but a good heuristic can give dramatic improvement
- Space? $O(b^m)$, keeps all nodes in memory
- Optimal? No

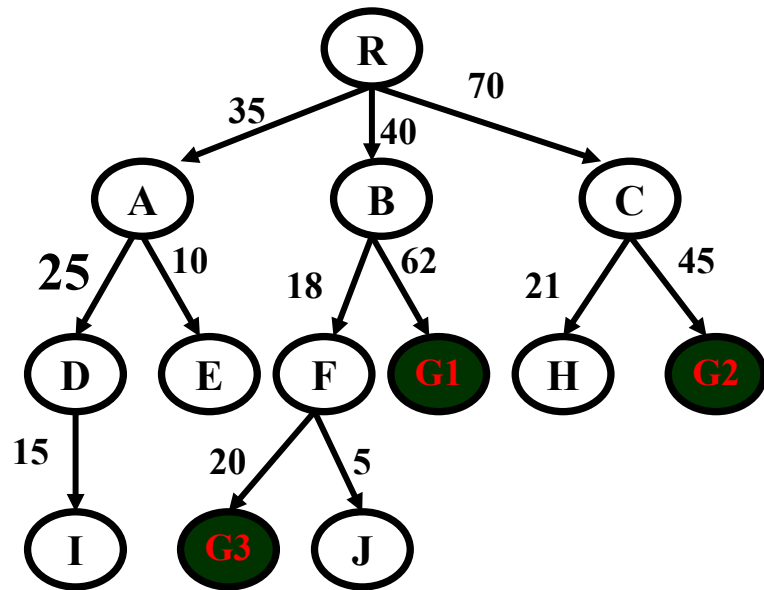
A* search

- **Idea:** avoid expanding paths that are already expensive
- Evaluation function $f(n) = g(n) + h(n)$ *where*
- $g(n)$ = cost so far to reach n
- $h(n)$ = estimated cost from n to goal
- $f(n)$ = estimated total cost of path through n to goal
- It tries to minimize the total path cost to reach into the goal at every node N .
- **Example one**

Indicate the flow of search to move from Awasa to Gondar using A*

Example Two

- Given the following tree structure, show the content of the open list and closed list generated by A* best first search algorithm



Heuristic

R	→	G	-----	100
A	→	G	-----	60
B	→	G	-----	80
C	→	G	-----	70
D	→	G	-----	65
E	→	G	-----	40
F	→	G	-----	45
H	→	G	-----	10
I	→	G	-----	20
J	→	G	-----	8
G ₁ , G ₂ , G ₃	→	G	-----	0

Admissible heuristics

- A heuristic $h(n)$ is **admissible** if for every node n , $h(n) \leq h^*(n)$, where $h^*(n)$ is the **true** cost to reach the goal state from n .
- An admissible heuristic **never overestimates** the cost to reach the goal, i.e., it is **optimistic**
- Example: $h_{SLD}(n)$ (never overestimates the actual road distance)
- **Theorem:** If $h(n)$ is admissible, A* using TREE-SEARCH is optimal

Find Admissible heuristics for the 8-puzzle?

7	2	4
5		6
8	3	1

Start State

	1	2
3	4	5
6	7	8

Goal State

- $h_1(n)$ = number of misplaced tiles
- $h_2(n)$ = total Manhattan distance (i.e., no. of squares from desired location of each tile). This is also called **city cap** distance
- $h_1(S) = ?$
- $h_2(S) = ?$

Admissible heuristics

E.g., for the 8-puzzle:

- $h_1(n)$ = number of misplaced tiles
- $h_2(n)$ = total Manhattan distance
(i.e., no. of squares from desired location of each tile)
- $h_1(S) = ?$ 8
- $h_2(S) = ?$ $3+1+2+2+2+3+3+2 = 18$

7	2	4
5		6
8	3	1

Start State

	1	2
3	4	5
6	7	8

Goal State

• Dominance

—If $h_2(n) \geq h_1(n)$ for all n (both admissible)

—then h_2 **dominates** h_1

— h_2 is better for search

Properties of A*

- Complete? Yes (unless there are infinitely many nodes with $f \leq f(G)$)
- Optimal? Yes (provided that the heuristic is admissible)
- Time?
 - In the best case (if the heuristic is the same as the actual cost), it is equivalent to the depth of the solution node (i.e. it is linear). $O(d)$
 - In the worst case, it is equivalent to the number of nodes which has f-value $\leq$ f-value of the solution node $O(b^{\text{ceiling}(C^*/\epsilon)})$ where C^* is the f value of the solution node
 - This shows, A* search is computationally efficient compared to Greedy best first search strategy

Properties of A*

- Space? Keeps all nodes in memory (exponential)
 - i.e. $O(b^m)$
 - This is again a limitation in the same way as we saw while discussing Greedy, Breadth and Depth first search
 - Hence, it is advisable to have modified version of such algorithm in which the modification minimizes the space complexity
 - There are two such modifications
 1. Iterative deepening A* (IDA*) search
 2. Simplified Memory Bound A* (SMA*) search

Iterative Deepening A* (IDA*) search

- Iterative deepening is a useful technique for reducing memory requirements.
- The same trick as iterative deepening depth first search is applied to A* for the same purpose.
- This time the limiting factor will be *f-value* rather than depth value.
- At first step, the root *f-value* will be used as a threshold and all the nodes following the root will be visited until:
 - The solution found
 - The node *f-value* exceeds the root

Iterative Deepening A* (IDA*) search

- If solution doesn't exist in this limit, the *f-value* threshold will be modified as the minimum value of a *visited node* whose *f-value* exceeds the previous threshold
- This process repeat until solution found or failure (due to absence of extra node with *f-value* that exceed the last threshold
- the algorithm is stated in the next slide

Iterative Deepening A* (IDA*) search

*Function **IDA***(problem) returns a solution sequence or failure*

local variable: ***f-limit**,* *the current f-COST limit*
 ***root**,* *the root node*

root ← **MAKE_NODE**(**INITIAL_STATE**[problem])

f-limit ← **f-COST**(*root*)

loop do

solution, f-limit ← **F_CONTOUR_SEARCH**(*root*, *f-limit*)

 if *solution* is NOT NULL *return solution*

 if *f-limit* = INFINITY *return failure*

End

Iterative Deepening A* (IDA*) search

Function ***F_CONTOUR_SEARCH***(*node*, *f-limit*) *returns*
solution sequence and a new ***f-COST*** limit

local variables:

next-f the *f_COST* limit for the next contour, initially
INFINITY

if ***f_COST***[*node*] > *f-limit* *return* null, ***f-COST***[*node*]

if GOAL_TEST[*problem*](***STATE***[*node*]) *return* SOLUTION(*node*), *f-limit*
for each node s in SUCCESSOR(*node*) *do*

solution, new-f $\leftarrow$ ***F_CONTOUR_SEARCH***(*node*, *f-limit*)

 if *solution* is NOT NULL *return* *solution*, *f-limit*

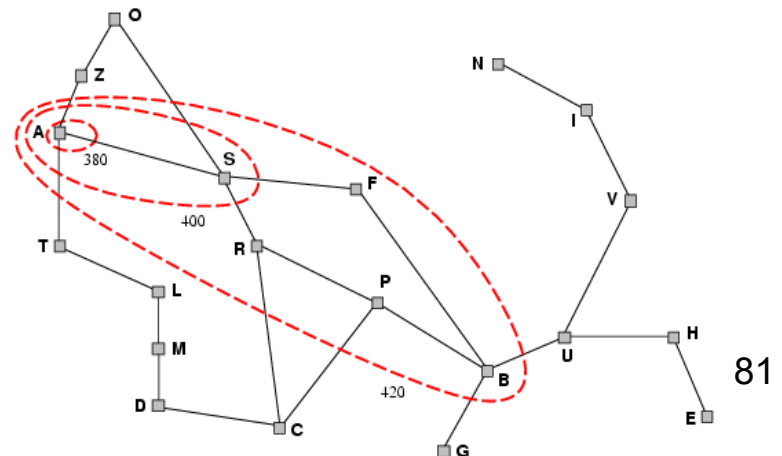
next-f $\leftarrow$ MIN(*next-f*, *new-f*)

END

return null, next-f

Iterative Deepening A* (IDA*) search

- IDA* is complete and optimal with the same caveat as A*
- Space and time complexity $\frac{bf^*}{\delta}$
- *Where b is the branching factor, f^* is the cost of the optimal solution and δ is the smallest step cost*
- IDA* has difficulty in more complex domains especially when the next f-value may only add one node than the nodes expanded in the previous iteration
- In this case, if A* expand N nodes, the IDA* in the worst case will expand $1 + 2 + \dots + N = O(N^2)$



Simplified Memory Bounded A* (SMA*) search

- SMA* algorithm can make use of all the available memory to carry out the search.
- SMA* has the following property
 - It will utilize whatever memory is made available to it
 - It avoids repeated states as far as its memory is sufficient to store the shallowest solution path
 - It is optimal if enough memory is available to store the shallowest optimal solution path.
 - Otherwise, it returns the best solution that can be reached with available memory
 - When enough memory is available for the entire search tree, the search is optimally efficient
- The next slide shows algorithm

Simplified Memory Bounded A* (SMA*) search

Function SMA(problem) return solution/failure*

Local variable: Queue, a queue of node ordered by f-value initially empty

Queue $\leftarrow$ MAKE-QUEUE(MAKE-MODE(INITIAL-STATE[problem]))

Loop do

if Queue is empty return failure

n $\leftarrow$ deepest least f-cost node in queue

if GOAL-TEST(n) return success

s $\leftarrow$ NEXT-SUCCESSOR(*n*)

if s is not a goal and is at maximum depth then (not goal and is a leaf)

f(s) $\leftarrow \infty$

else

f(s) $\leftarrow$ MAX(*f(n)*, *f(s)*)

if all n's successors have been generated then

update n's f-cost and those of its ancestors if necessary

if SUCCESSORS(n) are all in memory then remove n from Queue

if memory is full then

delete the shallowest, highest f-cost node in Queue

remove it from its parent's successor list

insert its parent on Queue if necessary

insert s on Queue

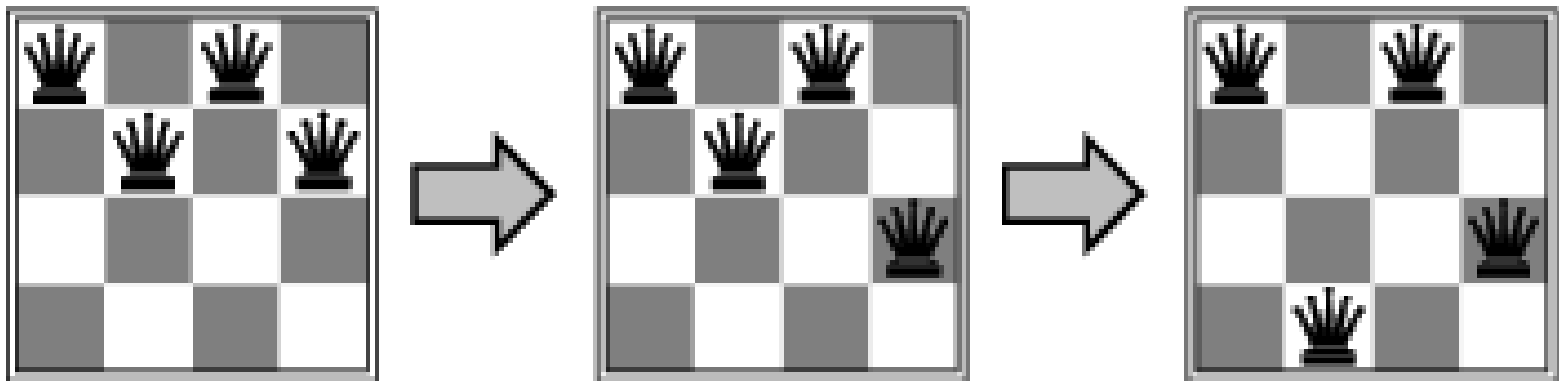
end

Iterative Improvement Algorithm (Local search algorithms)

- In many optimization problems, the **path** to the goal is irrelevant; the goal state itself is the solution
- State space = set of "complete" configurations
- Find configuration satisfying constraints, e.g., n-queens
- In such cases, we can use **local search algorithms**
- keep a single "current" state, try to improve it

Example: *n*-queens

- Put n queens on an $n \times n$ board with no two queens on the same row, column, or diagonal



Iterative Improvement Algorithm (Local search algorithms)

- There are two types of Iterative Improvement algorithms
 - **Hill climbing** if the evaluation function is quality
 - also called **Gradient Descent** if the evaluation function is a cost rather than a quality
 - **Simulated Annealing**

Hill-climbing search

- Tries to make changes that improve the current state cost
- The algorithm is given below
- It continually move in the direction of increasing value
- The node data structure maintain only records of state and evaluation cost

```
function HILL-CLIMBING(problem) returns a state that is a local maximum
  inputs: problem, a problem
  local variables: current, a node
                     neighbor, a node

  current ← MAKE-NODE(INITIAL-STATE[problem])
  loop do
    neighbor ← a highest-valued successor of current
    if VALUE[neighbor] ≤ VALUE[current] then return STATE[current]
    current ← neighbor
```

Hill-climbing (Gradient Descent) search

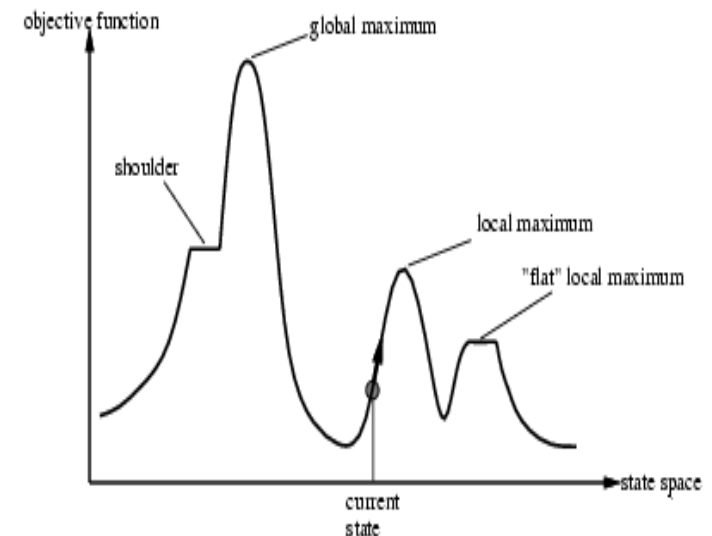
- Tries to make changes that improve the current state cost

```
function HILL-CLIMBING(problem) returns a state that is a local maximum
  inputs: problem, a problem
  local variables: current, a node
                   neighbor, a node

  current ← MAKE-NODE(INITIAL-STATE[problem])
  loop do
    neighbor ← a highest-valued successor of current
    if VALUE[neighbor] ≤ VALUE[current] then return STATE[current]
    current ← neighbor
```

Problem:

1. Depending on initial state, can get stuck in local maxima
2. Plateaux (after some progress the algorithm will make a random walk)
3. Ridges (a place where two sloppy sides meet). In this case the search may oscillate from side to side

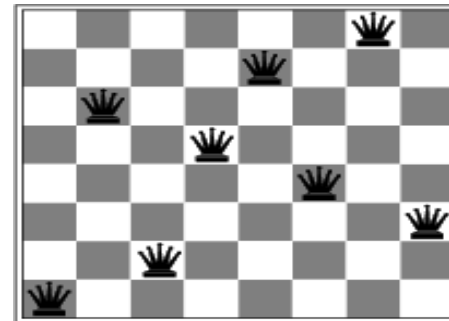


Hill-climbing search: 8-queens problem

- h = number of pairs of queens that are attacking each other, either directly or indirectly
- $h = 17$ for the above state

18	12	14	13	13	12	14	14
14	16	13	15	12	14	12	16
14	12	18	13	15	12	14	14
15	14	14	♚	13	16	13	16
♚	14	17	15	♚	14	16	16
17	♚	16	18	15	♚	15	♚
18	14	♚	15	15	14	♚	16
14	14	13	17	12	14	12	18

- A local minimum with $h = 1$
- *Improvement techniques*
 - *Random restart hill climbing for N iteration by saving the best state so far*



Simulated Annealing

- Idea: escape local maxima by allowing some "bad" moves but **gradually decrease** their frequency

```
function SIMULATED-ANNEALING(problem, schedule) returns a solution state
  inputs: problem, a problem
           schedule, a mapping from time to "temperature"
  local variables: current, a node
                   next, a node
                   T, a "temperature" controlling prob. of downward steps

  current ← MAKE-NODE(INITIAL-STATE[problem])
  for t ← 1 to ∞ do
    T ← schedule[t]
    if T = 0 then return current
    next ← a randomly selected successor of current
     $\Delta E \leftarrow \text{VALUE}[\textit{next}] - \text{VALUE}[\textit{current}]$ 
    if  $\Delta E > 0$  then current ← next
    else current ← next only with probability  $e^{\Delta E/T}$ 
```

Properties of simulated annealing search

- One can prove: If T decreases slowly enough, then simulated annealing search will find a global optimum with probability approaching 1
- Widely used in VLSI layout, airline scheduling, etc

Local beam search

- Keep track of k states rather than just one
- Start with k randomly generated states
- At each iteration, all the successors of all k states are generated
- If any one is a goal state, stop; else select the k best successors from the complete list and repeat.

Genetic algorithms

- Genetic Algorithms were invented to mimic some of the processes observed in natural evolution.
- Many people, biologists included, are astonished that life at the level of complexity that we observe could have evolved in the relatively short time suggested by the fossil record.
- The idea with GA is to use this power of evolution to solve optimization problems.
- The father of the original Genetic Algorithm was John Holland who invented it in the early 1970's.
- Genetic Algorithms (GAs) are adaptive heuristic search algorithm based on the evolutionary ideas of natural selection and genetics.

Genetic algorithms

- The basic techniques of the GAs follow the principles first laid down by Charles Darwin of "survival of the fittest."
- In nature, competition among individuals results in the fittest individuals dominating over the weaker ones.
- GA is better than conventional AI in that it is more robust.
- Unlike older AI systems, they do not break easily even if the inputs changed slightly, or in the presence of reasonable noise.
- Also, in searching a large state-space, multi-modal state-space, or n-dimensional surface, a genetic algorithm may offer significant benefits over more typical search of optimization techniques like linear programming, heuristic, depth-first, breath-first, and praxis.

Genetic algorithms

- GAs simulate the survival of the fittest among individuals over consecutive generation for solving a problem.
- Each generation consists of a population of character strings that are analogous to the chromosome that we see in our DNA.
- Each individual represents a point in a search space and a possible solution.
- The individuals in the population are then made to go through a process of evolution.
- GAs are based on an analogy with the genetic structure and behavior of chromosomes within a population of individuals using the following foundations:

Genetic algorithms

Basic GA foundations

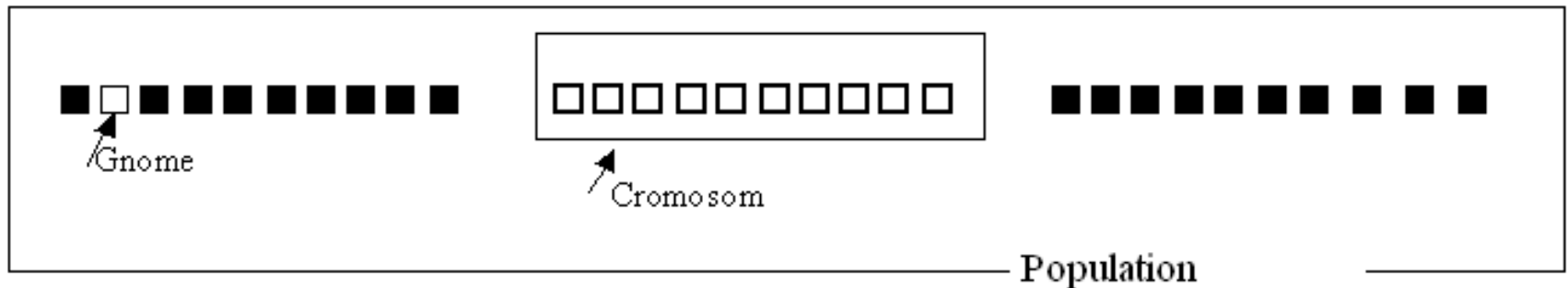
- Individuals in a population compete for resources and mates.
- Those individuals most successful in each 'competition' will produce more offspring than those individuals that perform poorly.
- Genes from 'good' individuals propagate throughout the population so that two good parents will sometimes produce offspring that are better than either parent.
- Thus each successive generation will become more suited to their environment.

Genetic algorithms Search Space

- A population of individuals are maintained within **search space** for a GA, each representing a possible solution to a given problem.
- Each individual is coded as a finite length vector of components, or variables, in terms of some alphabet, usually **binary** alphabet **{0,1}**.
- To continue the genetic analogy these individuals are likened to chromosomes and the variables are analogous to genes.
- Thus a chromosome (solution) is composed of several genes (variables).
- A **fitness score** is assigned to each solution representing the abilities of an individual to 'compete'.
- The individual with the optimal (or generally near optimal) fitness score is sought.

Genetic algorithms Search Space

- The GA aims to use selective 'breeding' of the solutions to produce 'offspring' better than the parents by combining information from the chromosomes.



Genetic algorithms Search Space

- The GA maintains a population of n chromosomes (solutions) with associated fitness values.
- Parents are selected to mate, on the basis of their fitness, producing offspring via a reproductive plan.
- Highly fit solutions are given more opportunities to reproduce, so that offspring inherit characteristics from each parent.
- Since the population is kept at a static size, individuals in the population die and are replaced by the new solutions, eventually creating a new generation once all mating opportunities in the old population have been exhausted.
- In this way it is hoped that over successive generations better solutions will thrive while the least fit solutions die out.
- Eventually, once the population has converged and is not producing offspring noticeably different from those in previous generations, the algorithm itself is said to have converged to a set of solutions to the problem at hand.

Genetic algorithms Implementation Details

- After an initial population is randomly generated, the algorithm evolves the through three operators:
 - 1.**selection** which equates to survival of the fittest;
 - 2.**crossover** which represents mating between individuals;
 - 3.**mutation** which introduces random modifications.

1. Selection Operator

- key idea: give preference to better individuals, allowing them to pass on their genes to the next generation.
- The goodness of each individual depends on its fitness.
- Fitness may be determined by an objective function or by a subjective judgment.

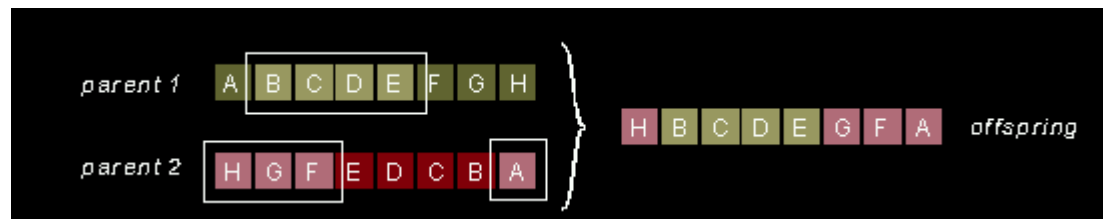
2. Crossover Operator

- Prime distinguished factor of GA from other optimization techniques
- Two individuals are chosen from the population using the selection operator

Genetic algorithms Implementation Details

2. Crossover Operator (cont)

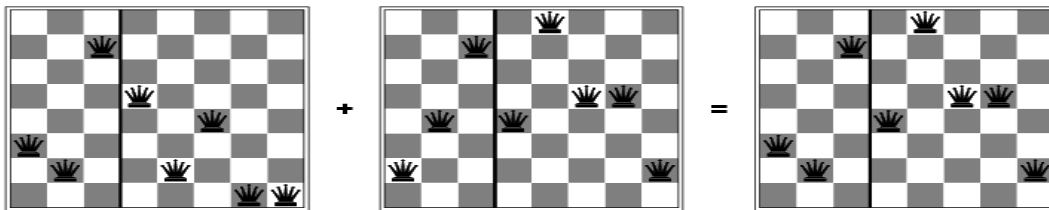
- A crossover site along the bit strings is randomly chosen
- The values of the two strings are exchanged up to this point
- If $S1=000000$ and $s2=111111$ and the crossover point is 2 then $S1'=110000$ and $s2'=001111$
- The two new offspring created from this mating are put into the next generation of the population
- By recombining portions of good individuals, this process is likely to create even better individuals



Genetic algorithms Implementation Details

3. Mutation Operator

- With some low probability, a portion of the new individuals will have some of their bits flipped.
- Its purpose is to maintain diversity within the population and inhibit premature convergence.
- Mutation alone induces a random walk through the search space
- Mutation and selection (without crossover) create a parallel, noise-tolerant, hill-climbing algorithms
- Example:



Genetic algorithms Implementation Details

3. Mutation Operator

• **Fitness function: number of non-attacking pairs of queens (min = 0, max = $8 \times 7/2 = 28$)**

• **$24/(24+23+20+11) = 31\%$; $23/(24+23+20+11) = 29\%$ etc**



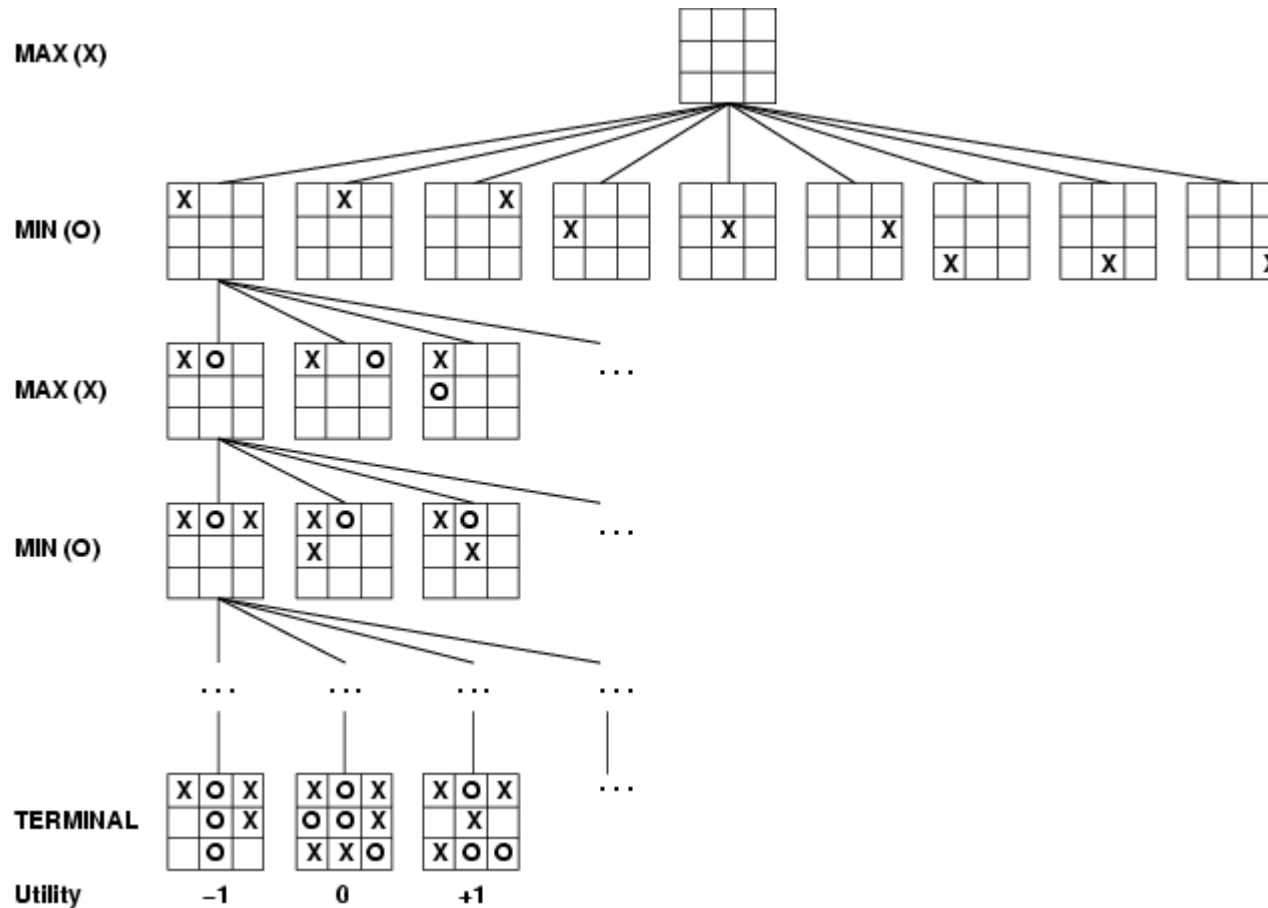
Game playing (Adversarial Search)

- **Outlines**
 - How to make optimal decisions in two player game
 - MinMax algorithm
 - α - β pruning algorithm
- In Game theory there are always at least two agents that participate.
- There may be different groups that participate in game where each of them go for **win** or **maximize** the objective
- In this topic, we focus on only two player game:
 - Player1 wants to maximize his objective function at the end of the game
 - Player2 (**opponent**): wants to minimize player1 objective function

Game playing (Adversarial Search)

- Opponent always introduce uncertainty because one never knows what action the opponent may choose
- This unpredictable nature of game playing makes it different from search problem.
- In most cases, game playing has very large branching factor which will have a direct impact on the implementation time and space complexity
- Example: Tic-Tac-Toe

Game tree (2-player, deterministic, turns)



- **Components**
 - Initial state (environment + whose turn to move)
 - Operators (defines legal move to the agent)
 - Terminal test
 - Utility function (payoff function)

Minimax Algorithm

- Perfect play for deterministic games
- Idea: choose move to position with highest **Minimax value**
= best achievable payoff against best play
- E.g., 2-ply game:
- The algorithm consists of five steps
 1. Generate the whole tree
 2. Apply the utility function to each terminal state to get its value
 3. Determine the utility of upper state using the lower states
 4. Continue upward until the root
 5. Max should choose the best play

Minimax algorithm

function MINIMAX-DECISION(*state*) *returns an action*

$v \leftarrow \text{MAX-VALUE}(\text{state})$

return the *action* in **SUCCESSORS**(*state*) with value *v*

function MAX-VALUE(*state*) *returns a utility value*

if **TERMINAL-TEST**(*state*) **then return** **UTILITY**(*state*)

$v \leftarrow -\infty$

for *a, s* **in** **SUCCESSORS**(*state*) **do**

$v \leftarrow \text{MAX}(v, \text{MIN-VALUE}(s))$

return *v*

function MIN-VALUE(*state*) *returns a utility value*

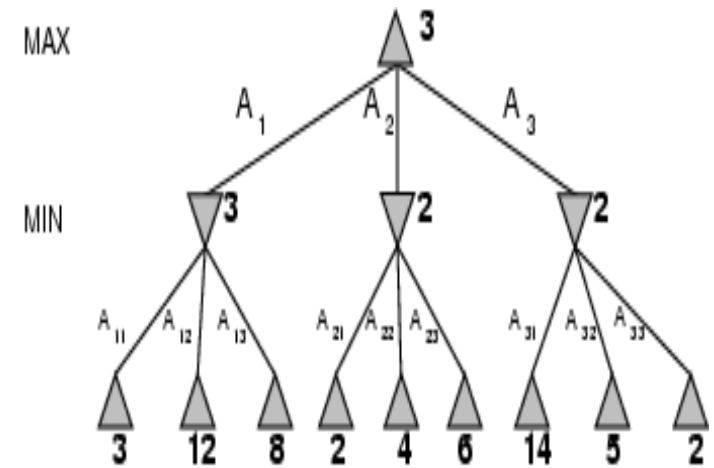
if **TERMINAL-TEST**(*state*) **then return** **UTILITY**(*state*)

$v \leftarrow \infty$

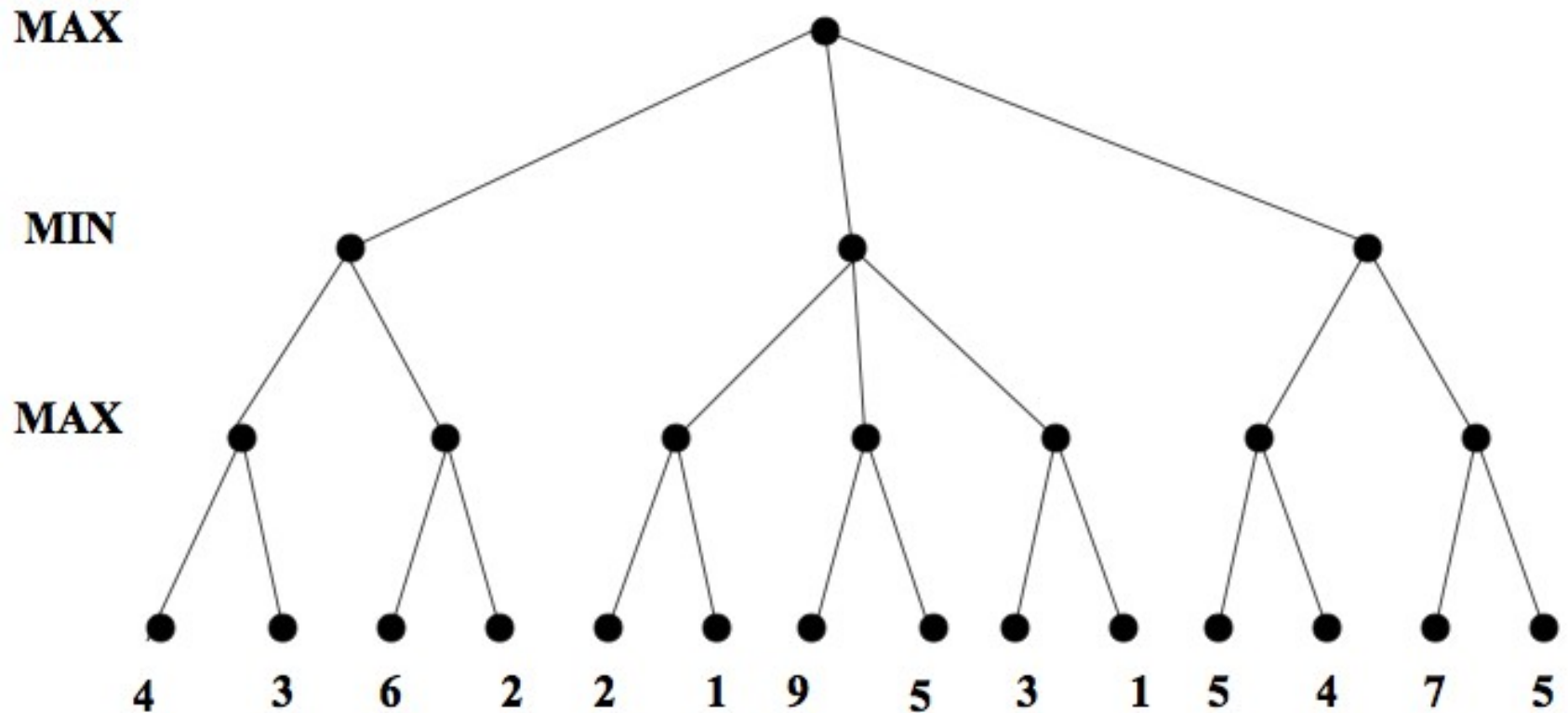
for *a, s* **in** **SUCCESSORS**(*state*) **do**

$v \leftarrow \text{MIN}(v, \text{MAX-VALUE}(s))$

return *v*



Minimax. Example



Properties of minimax

- Complete? Yes (if tree is finite)
- Optimal? Yes (against an optimal opponent)
- Time complexity? $O(b^m)$
- Space complexity? $O(bm)$ (depth-first exploration)

α - β pruning

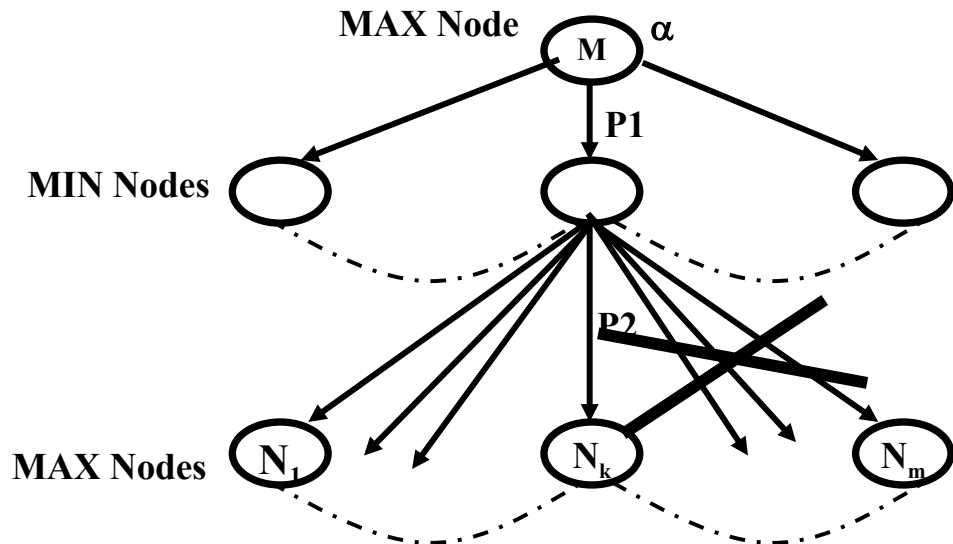
- The MiniMax algorithm works well in almost any game problems.
- But its time complexity is very very discouraging to problems of relatively larger depth and branching factor.
- For example, in chess game the branching factor is around 35 and if the game need around 5 move each (total 10 moves) then the time complexity becomes 35^{10} .
- If we assume our computer process 100000 per second, it will take several years.
- Therefore, we should think a better algorithm that optimizes the search
- α - β Pruning is one specific pruning technique used in game theory
- By pruning we mean removing paths which will not take the agent to a better solution.

α - β pruning

- **Alpha (α)** *minimal score that player **MAX** is guaranteed to attain.*
- **Beta (β)** *maximum score that player **MAX** can hope to obtain against a sensible opponent.*

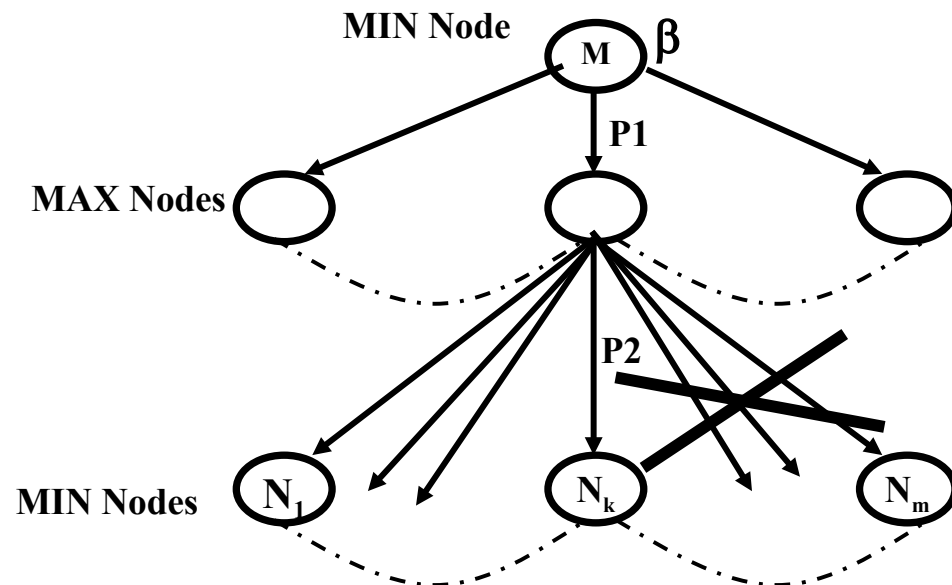
α - β pruning algorithm description

- **Case one: pruning via calling MIN function**
- Consider M is a node for MAX and it has guaranteed α using all the paths to the left of P1 and assume the utility of N_i for $i < K$ is greater than α . However utility of $N_k < \alpha$. This shows if MAX choose to apply action P1, then MIN will choose P2 that minimizes MAX utility which MAX don't want at all. Therefore the moment this situation happen, no need to investigate all the sub trees with roots N_i where $k < i \leq m$



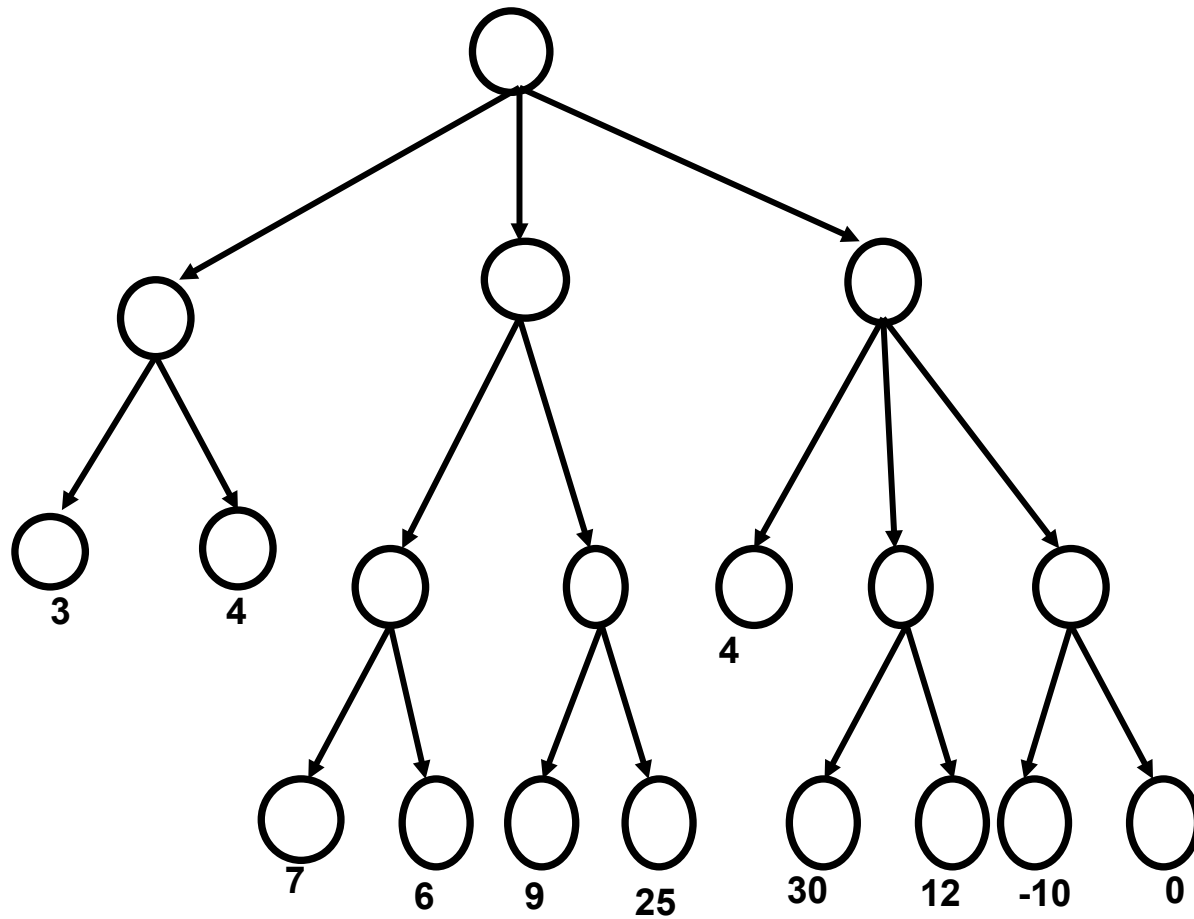
α - β pruning algorithm description

- **Case two: pruning via calling MAX function**
- Consider M is a node for MIN and it knows that MAX could obtain β using all the paths to the left of P1 and assume the utility of N_i for $i < K$ is less than β . However utility of $N_k > \beta$. This shows if MIN choose to apply action P1, then MAX will choose P2 that maximizes MAX utility which MIN don't want at all. Therefore the moment this situation happen, no need to investigate all the sub trees with roots N_i where $k < i \leq m$



Example:

Show the utility of each of nodes and prune unnecessary nodes using α - β pruning algorithm for the following state space tree



The α - β algorithm

function ALPHA-BETA-SEARCH(*state*) *returns an action*

inputs: *state*, current state in game

$v \leftarrow \text{MAX-VALUE}(\text{state}, -\infty, +\infty)$

return the *action* in SUCCESSORS(*state*) with value *v*

function MAX-VALUE(*state*, α , β) *returns a utility value*

inputs: *state*, current state in game

α , the value of the best alternative for MAX along the path to *state*

β , the value of the best alternative for MIN along the path to *state*

if TERMINAL-TEST(*state*) **then return** UTILITY(*state*)

$v \leftarrow -\infty$

for *a, s* in SUCCESSORS(*state*) **do**

$v \leftarrow \text{MAX}(v, \text{MIN-VALUE}(s, \alpha, \beta))$

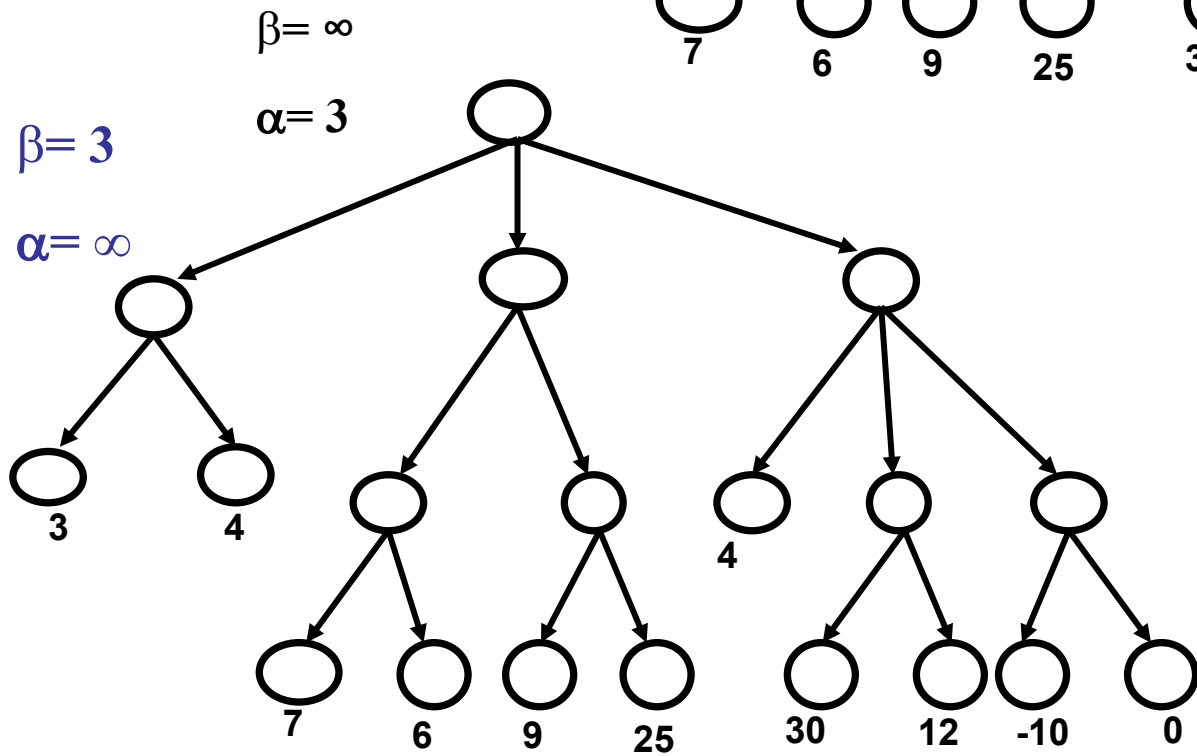
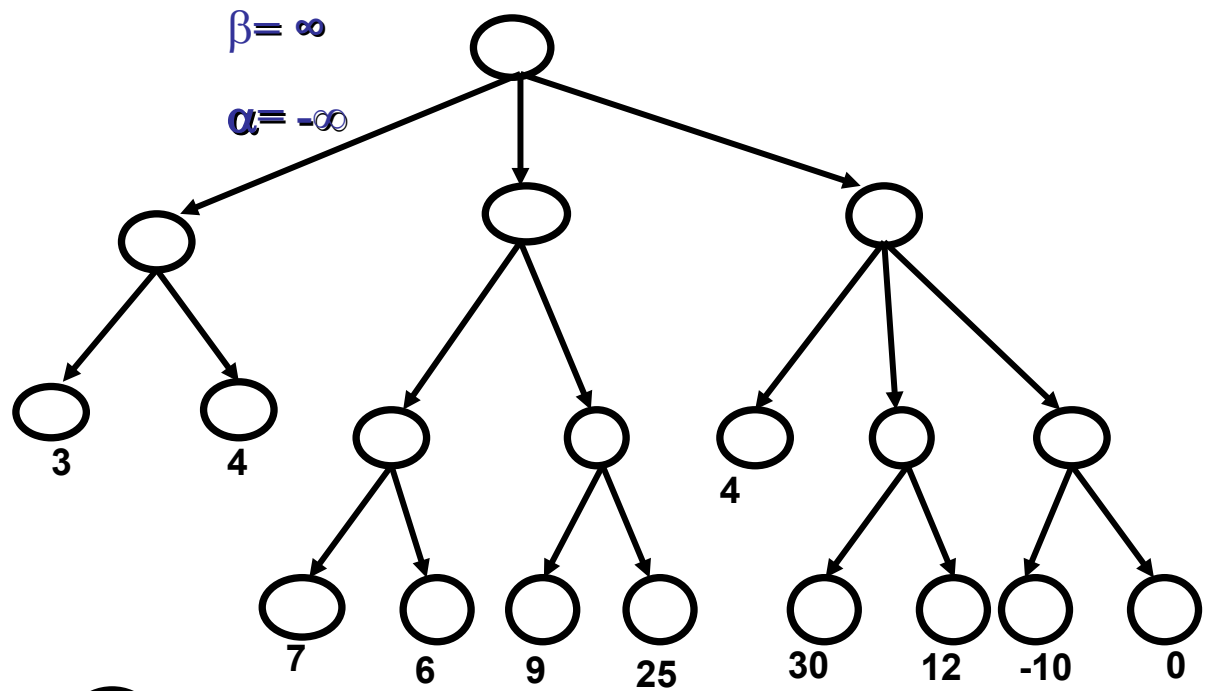
if $v \geq \beta$ **then return** *v*

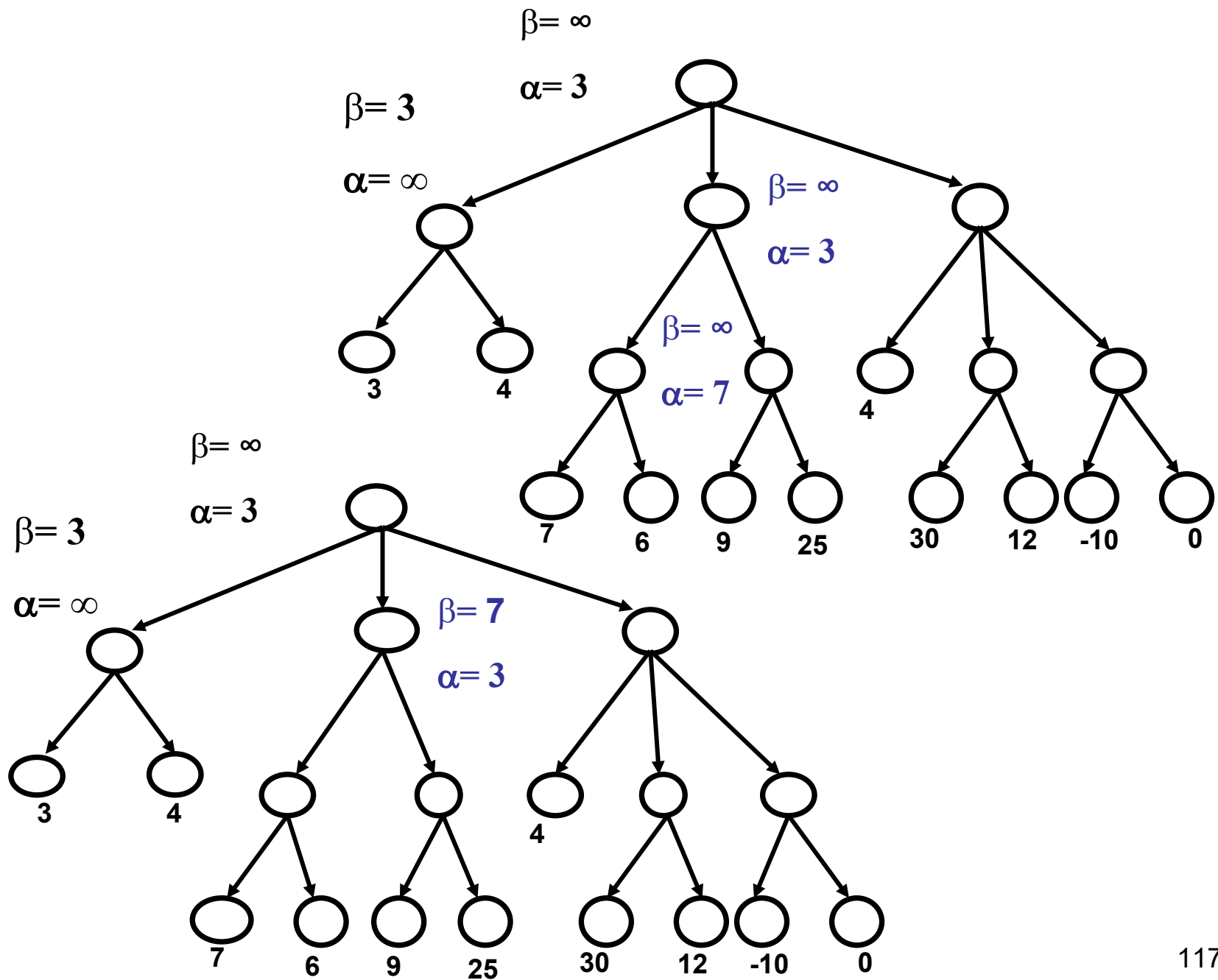
$\alpha \leftarrow \text{MAX}(\alpha, v)$

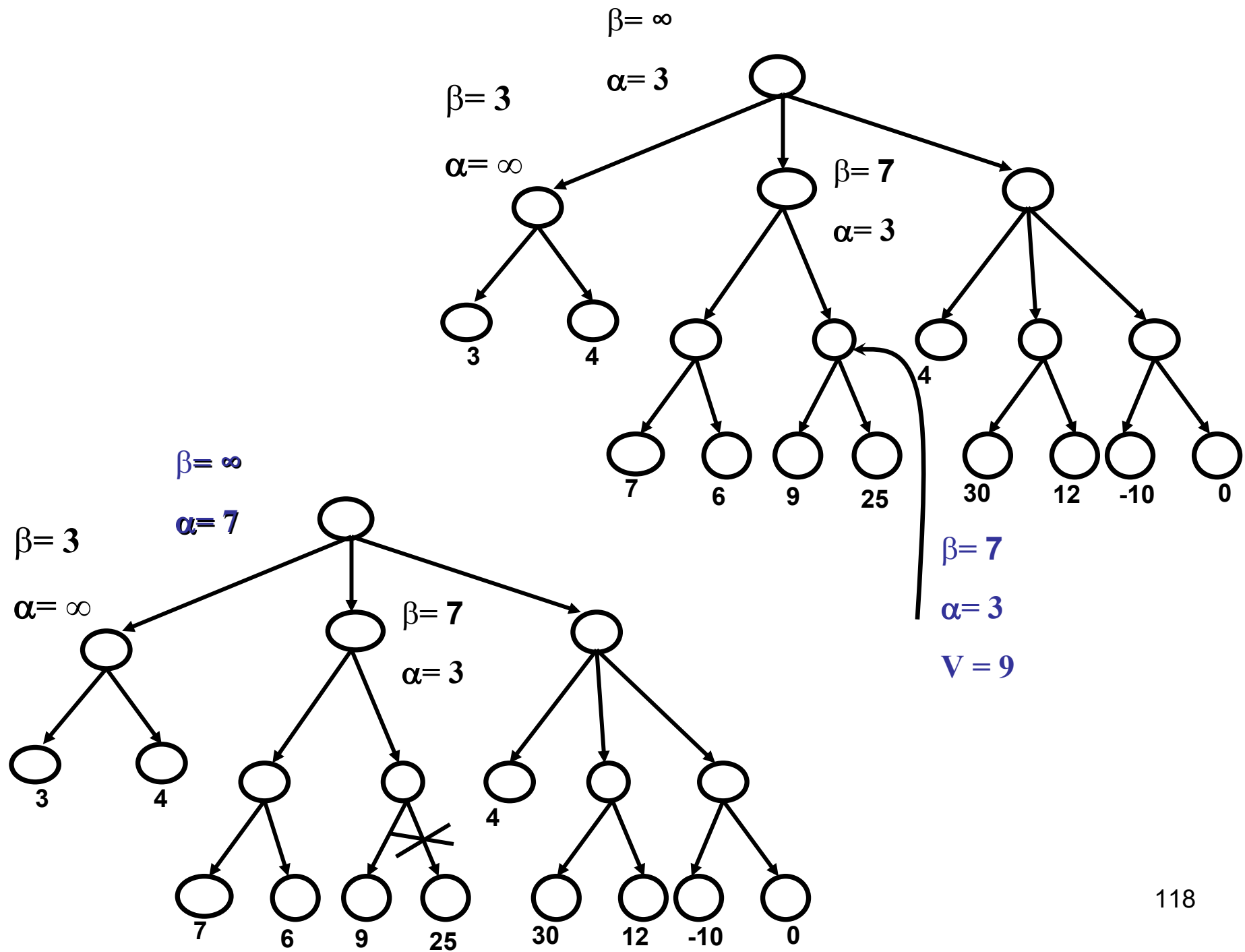
return *v*

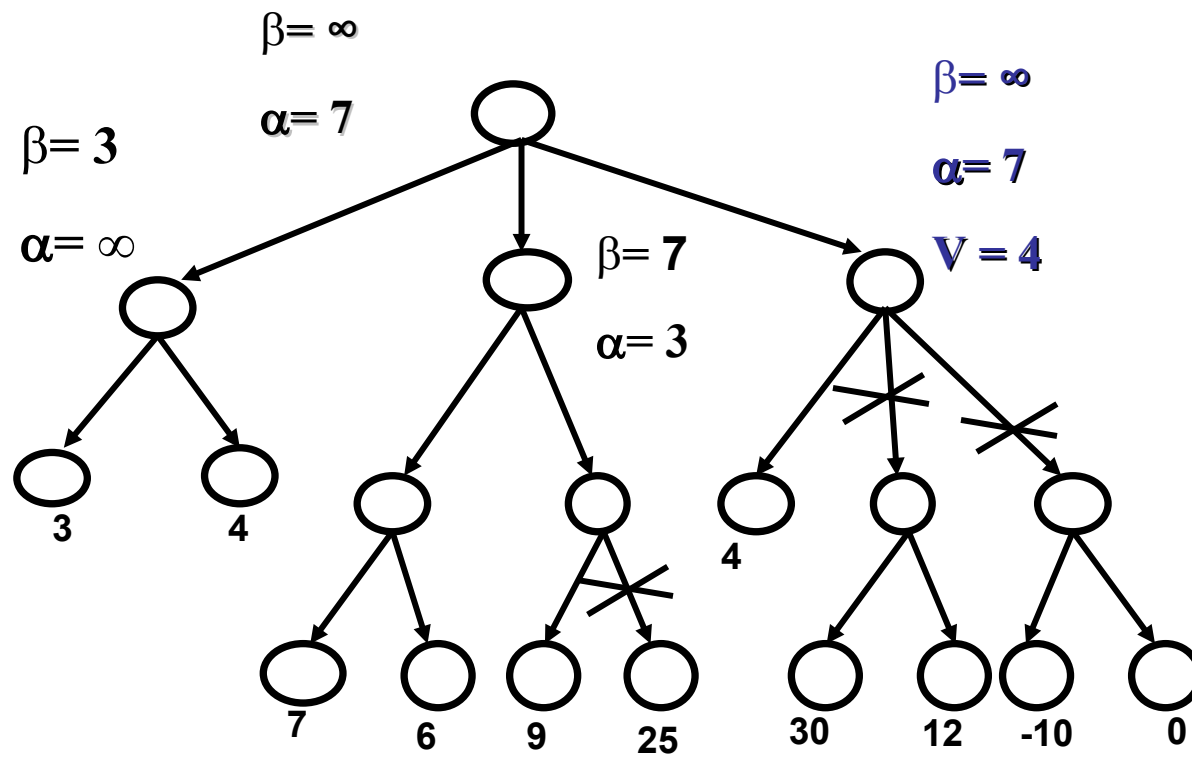
The α - β algorithm

```
function MIN-VALUE(state,  $\alpha$ ,  $\beta$ ) returns a utility value  
  inputs: state, current state in game  
            $\alpha$ , the value of the best alternative for MAX along the path to state  
            $\beta$ , the value of the best alternative for MIN along the path to state  
  
  if TERMINAL-TEST(state) then return UTILITY(state)  
   $v \leftarrow +\infty$   
  for  $a, s$  in SUCCESSORS(state) do  
     $v \leftarrow \text{MIN}(v, \text{MAX-VALUE}(s, \alpha, \beta))$   
    if  $v \leq \alpha$  then return  $v$   
     $\beta \leftarrow \text{MIN}(\beta, v)$   
  return  $v$ 
```









Properties of α - β

- Pruning **does not** affect final result
- Good move ordering improves effectiveness of pruning
- With "perfect ordering," time complexity = $O(b^{m/2})$
 - **doubles** depth of search
- **Why is it called α - β ?**
 - α is the value of the best (i.e., highest-value) choice found so far at any choice point along the path for *max*
 - If v is worse than α , *max* will avoid it
 - prune that branch
 - Define β similarly for *min*

